SUNWAY UNIVERSITY | Faculty of ENGINEERING & TECHNOLOGY

CAPSTONE PROJECT 2

Final Project Report

ZY-Path: A GUI-Based AI Histopathology Image Classification Platform

by

LI ZI YAO

24018160

Bachelor of Software Engineering (Hons)

Supervisor: Dr Wan Siti Halimatul Munirah Wan Yahya @ Wan Ahmad

Semester: [TO CONFIRM]

Date: [TO CONFIRM]

School of Computing and Artificial Intelligence
Faculty of Engineering and Technology
Sunway University

Acknowledgements

I would like to thank my supervisor, Dr Wan Siti Halimatul Munirah Wan Yahya @ Wan Ahmad, for the guidance provided throughout this capstone project. The project developed from the CP1 proposal into an implemented desktop application through repeated review of the analysis workflow, model integration and user interface.

I also acknowledge the researchers and maintainers whose open-source work makes this project possible, particularly the PyTorch, torchvision, Qt/PyQt, Pillow, NumPy, Matplotlib, SQLite and fpdf2 communities. Their software and documentation provided the foundation for implementing and testing ZY-Path. The published digital-pathology and usability research discussed in this report informed the project's scope and evaluation design.

Abstract

Histopathology image analysis increasingly uses deep learning, but applying a trained model still requires users to manage compatibility, preprocessing, result interpretation and analysis records. This project develops ZY-Path, a local desktop application that brings these activities into a graphical workflow for education and exploratory research. The implementation uses PyQt5 for the interface, PyTorch for CPU inference and SQLite for persistent history. It supports compatible pretrained model files, whole-image classification, explicit tiled classification, segmentation, linked original/result inspection and PNG/PDF export. The built-in classification integration follows a custom nine-class DenseNet contract: RGB float32 input, raw255 pixel values, a 224 × 224 network input and a fixed model class order. Tiled analysis classifies non-overlapping source crops and reports area-weighted estimated coverage, distinctly from pixel segmentation. Original-image tile dimensions are editable; the 512 × 512 starting value is not claimed to match an unverified training scale. The segmentation integration uses a separate UPerNet/Swin Base pathway with its own preprocessing, labels and overlapping-tile fusion. Model validation and inference run outside the GUI thread, while cooperative cancellation, task-specific model defaults and recorded rerun parameters support a controlled workflow. At the documented implementation revision, automated verification recorded 355 passed tests, one failed test caused by absent real model weights and one collection error caused by a missing training script. These results establish substantial software-level evidence but do not establish diagnostic accuracy. User acceptance testing and independent model evaluation have not yet been completed; their protocols, instruments and result templates are supplied without invented findings. ZY-Path's contribution is an inspectable, record-oriented desktop integration, rather than a new neural architecture, a whole-slide processing platform or a clinically validated diagnostic system.

Keywords: digital pathology; desktop application; PyQt5; PyTorch; tiled classification; semantic segmentation; reproducible workflow.

Table of Contents

Acknowledgements.....i
Abstract.....ii
List of Figures.....vi
List of Tables.....vii
Chapter 1: Introduction.....1
1.1 Background and Motivation.....1
1.2 Problem Formulation.....2
1.2.1 Problem Statement.....3
1.3 Hypothesis.....4
1.4 Project Objectives.....4
1.5 Scope of Work.....5
1.6 Research Significance.....7
Chapter 2: Literature Review.....8
2.1 Introduction.....8
2.2 Thematic Analysis of Related Research.....8
2.2.1 Digital histopathology and the unit of analysis.....8
2.2.2 Classification architectures and pretrained deployment.....9
2.2.3 Semantic segmentation and multiscale context.....10
2.2.4 Tiled classification, coverage and confidence.....10
2.2.5 No-code interaction and transparent constraints.....11
2.2.6 Persistence and reproducible workflows.....11
2.2.7 Human-centred design and evaluation.....12
2.3 Review of Existing Solutions and Platforms.....13
2.3.1 QuPath.....13
2.3.2 Slideflow and Slideflow Studio.....13
2.3.3 WSInfer and QuPath integration.....13
2.3.4 Cytomine.....14
2.3.5 Commercial environments.....14
2.4 Comparative Evaluation of Existing Systems.....14
2.5 Identified Research Gaps and Project Positioning.....15
2.6 Technological Infrastructure and Implementation Considerations.....16
2.7 Synthesis of Findings and Project Implications.....17
2.8 Summary.....18
Chapter 3: Methodology.....19
3.1 Overview of System Methodology.....19
3.2 Functional and Non-Functional Requirements.....21
3.2.1 Functional Requirements.....21
3.2.2 Non-Functional Requirements.....22

3.2.3 Ethical Considerations....23
3.2.4 Accessibility Compliance....23
3.3 Methodologies, Tools, and Techniques....24
3.3.1 Model representations and executable validation....24
3.3.2 Classification model and training-reference audit....25
3.3.3 Classification preprocessing and resize provenance....26
3.3.4 Explicit classification modes and crop geometry....26
3.3.5 Class probabilities, maps and estimated coverage....27
3.3.6 Segmentation pathway and label restoration....28
3.3.7 Background execution and cancellation....29
3.3.8 History schema, artifacts and rerun isolation....30
3.3.9 Export and local model defaults....31
3.4 User Interface (UI) and User Experience (UX) Design....32
3.4.1 Principles and information hierarchy....32
3.4.2 Spatial results and progressive disclosure....33
3.4.3 Control states, settings and help....33
3.4.4 Responsive windows and platform limits....34
3.5 Unified Modeling Language (UML) Diagrams....34
3.5.1 Use Case Diagram....34
3.5.2 Sequence Diagram....35
3.5.3 Class Diagram....36
3.5.4 Activity Diagram....38
3.6 Testing Methodology....39
3.6.1 Development Testing....39
3.6.2 Module Testing and Fallback Handling....39
3.6.3 Manual System Testing....40
3.6.4 User Acceptance Testing (UAT)....40
3.6.5 Testing Summary....41
3.6.6 Requirements Traceability Matrix....42
3.7 Risk Management and Mitigation Strategies....43
3.8 Summary....45
Chapter 4: Results and Discussion....46
4.1 Introduction....46
4.2 System Implementation Results....46
4.2.1 Implemented workflow and scope....46
4.2.2 Classification results and spatial inspection....47
4.2.3 Segmentation results....49
4.2.4 History, rerun and export....49
4.2.5 Settings, cancellation and interface polish....50
4.3 Functional Testing Results....52
4.3.1 Recorded automated results....52
4.3.2 Challenges, resolutions and evidence....54

4.3.3 Manual and predictive validation status....55
4.4 User Acceptance Testing Results....55
4.4.1 Participant Demographics....55
4.4.2 Quantitative Results....56
4.4.3 Feature Usage and Ratings....57
4.4.4 System Performance and Acceptability....57
4.4.5 Qualitative Feedback Analysis....58
4.5 Discussion....58
4.5.1 Achievement of Research Objectives....58
4.5.2 Comparison with Existing Solutions....59
4.5.3 Limitations and Constraints....60
4.5.4 Implications for Future Development....61
4.6 Summary....61
Chapter 5: Conclusion and Future Work....63
5.1 Conclusion....63
5.2 Future Work and Recommendations....64
5.3 Research Contributions....64
5.4 Summary....65
References....66
Appendices....68
Appendix A: Project Timeline and Planning....68
A.1 Proposal baseline....68
A.2 Implementation and outstanding milestones....73
A.3 Completion sequence....74
Appendix B: User Acceptance Testing Details....74
B.1 Participant information and consent briefing....74
B.2 Background questions....75
B.3 Standard task script and observation rules....75
B.4 System Usability Scale questionnaire....76
B.5 Feature-specific ratings and open questions....77
B.6 Scoring, missing data and reporting....77
Appendix C: Implementation and Evidence Index....78
C.1 Source baseline....78
C.2 Evidence interpretation....79
Appendix D: Manual Windows Acceptance and Screenshot Checklist....79
D.1 Native acceptance record....79
D.2 Required figure replacements....80
Appendix E: Independent Model Evaluation Protocol....80
E.1 Classification....80
E.2 Segmentation....81
E.3 Runtime and deployment....81

List of Figures

Figure 1.1. Representative H&E colorectal tissue image (evidence pending; replace this reserved area with the actual authorised capture).....1

Figure 2.1. Three output meanings: whole-image classification, source-tile estimates and pixel segmentation. Author-created conceptual diagram; it contains no model accuracy results.....11

Figure 3.1. Implemented logical architecture and artifact flow. Source: author, based on the reviewed Python modules.....20

Figure 3.2. Annotated final workflow wireframe. This author-created design diagram summarises implemented regions and is not a Windows runtime screenshot.....32

Figure 3.3. Use case diagram for the implemented local analysis workflow. File/model trust and clinical interpretation remain external responsibilities.....35

Figure 3.4. Sequence diagram for a run and its persistence boundary. Cancellation is acknowledged after a safe checkpoint; artifact-save failures must not be represented as successful saved records.....36

Figure 3.5. Implementation-level class diagram. Relationships represent principal ownership or dependency, not a generated inventory of every method.....37

Figure 3.6. Activity diagram from parameter selection to completed persistence, with safe cancellation and error exits.....38

Figure 4.1. Final Windows application overview (evidence pending; replace this reserved area with the actual authorised capture).....47

Figure 4.2. Tiled classification interface with source/result comparison and the revised result toggles. Linux/Qt capture using synthetic source/output data; not a real-weight or diagnostic result.....48

Figure 4.3. Real-weight single-image and tiled classification evidence (evidence pending; replace this reserved area with the actual authorised capture).....48

Figure 4.4. Windows segmentation result and aligned comparison (evidence pending; replace this reserved area with the actual authorised capture).....49

Figure 4.5. History and recorded-parameter rerun (evidence pending; replace this reserved area with the actual authorised capture).....50

Figure 4.6. Exported report and companion artifacts (evidence pending; replace this reserved area with the actual authorised capture).....50

Figure 4.7. Implemented Guide & Support page with fixed Contents navigation and numbered guidance. Linux/Qt development capture; contact/guide content is interface evidence rather than UAT feedback.....51

Figure 4.8. English history-calendar popup under a simulated Chinese default locale. Linux/Qt capture; the selected date is test input.....51

Figure 4.9. Cancellation and per-task model defaults on Windows (evidence pending; replace this reserved area with the actual authorised capture).....52

Figure 4.10. Recorded pytest command/outcome excerpt and scope notes. Reproduced from saved logs for the reviewed revision; not a model-performance chart.....53

List of Tables

Table 2.1. Scope-based comparison of pathology platforms....15
Table 3.1. Functional requirements and acceptance conditions....21
Table 3.2. Non-functional requirements and verification boundaries....22
Table 3.3. Classification class order and fixed display palette v1....28
Table 3.4. Built-in segmentation labels and display colours....29
Table 3.5. Implemented persistent data structures....30
Table 3.6. Requirements traceability matrix....42
Table 3.7. Risk register and residual mitigations....44
Table 4.1. Recorded automated verification outcomes at revision 1f3e01e....52
Table 4.2. Implementation challenges and verified resolutions....54
Table 4.3. UAT participant profile — awaiting actual responses....55
Table 4.4. UAT quantitative summary — awaiting valid data....56
Table 4.5. Feature ratings — awaiting actual use and responses....57
Table 4.6. Objective achievement and outstanding evidence....58
Table A.1. Original CP1 planning baseline, retained from the submitted proposal....68
Table A.2. Original CP2 planning baseline, retained from the submitted proposal....70
Table A.3. Evidence-based milestone status....73
Table B.1. UAT tasks and success criteria....75
Table C.1. Principal evidence sources....78
Table D.1. Final Windows acceptance cases....79

Chapter 1: Introduction

1.1 Background and Motivation

Histopathology plays a key role in modern medical care by offering the examination of tissue samples at the microscopic level to aid in diagnosing disease, in predicting disease outcome and in planning therapy. New methods to analyze histopathological morphology have been focused on reducing the manual work of pathologists, which is traditionally the main means for analyzing histopathological features and suffers from intra- and inter-observer variability. In parallel with the fast development of digital pathology and artificial intelligence, medical images have been digitized and analysis/computation on images has been widely applied to enhance productivity and aid clinical decision-making.

Histopathology is a reference method for diagnosing many diseases, including malignancies. In haematoxylin and eosin (H&E) images, the staining of nuclei and cytoplasmic structures provides morphological information that trained professionals interpret in context. Digital imaging makes these observations available for storage, teaching and computational analysis. A model's output, however, is an inference from image appearance and must not be equated with a verified clinical diagnosis (Litjens et al., 2017; Kather et al., 2019).

EVIDENCE TO INSERT — Representative H&E colorectal tissue image

Insert an authorised, de-identified image used in this project, with dataset/sample identifier, source credit and known scale if available. This replaces the proposal's illustrative tissue image only if its provenance cannot be confirmed. Do not label a generated test pattern as histology.

Figure 1.1. Representative H&E colorectal tissue image (evidence pending; replace this reserved area with the actual authorised capture).

Convolutional neural networks support classification and segmentation through learned visual representations. DenseNet provides feature reuse through dense connectivity, while encoder–decoder and multiscale architectures support spatial prediction (Huang et al., 2017; Ronneberger et al., 2015; Xiao et al., 2018). Their practical use also depends on apparently small implementation choices: the channel order, numerical range, interpolation method, class order and image scale must agree with the model's requirements. A visually convincing result does not by itself demonstrate that these choices are correct.

Existing pathology platforms already offer sophisticated graphical inspection and analysis. QuPath provides an open-source image-analysis environment; Slideflow includes real-time whole-slide visualisation through Slideflow Studio; and WSInfer integrates reusable pretrained models with QuPath (Bankhead et al., 2017; Dolezal et al., 2024; Kaczmarzyk et al., 2024). The motivation for ZY-Path is therefore a narrower workflow need: to organise compatible model loading, source-image analysis, understandable result inspection and local inference records in a compact desktop application. The project does not depend on claiming that existing platforms have no GUI or cannot support reproducible work.

The software is aimed at students, educators and biomedical researchers who wish to inspect model behaviour without writing inference code. It separates two questions that are often confused: whether an application runs the intended calculation, and whether the resulting predictions are accurate for a clinical or research population. The former can be investigated with controlled software tests. The latter requires appropriately labelled independent data, known sampling conditions and a valid evaluation design.

1.2 Problem Formulation

The application-level problem is to turn a compatible trained model and a raster pathology image into a traceable analysis result through a graphical workflow. An analysis can be represented as R = f(I, M, T, P, G), where I is the source image, M the model, T the selected task and classification mode, P the preprocessing contract, and G the spatial sampling geometry. Results should retain enough of these inputs to explain what was run and to support a subsequent rerun. Display choices, such as overlay opacity, should change presentation rather than the underlying prediction or coverage calculation.

This formulation extends the CP1 image-level workflow to explicit tiled classification. Whole-image classification produces one probability vector for the complete resized image. Tiled classification produces one vector per crop and a spatial class map. Segmentation produces a pixel-label map through a separate model pathway. Treating these outputs as interchangeable would misrepresent their evidence, particularly when a tile-level coverage estimate is mistaken for a pixel-accurate boundary.

The original research questions remain relevant to this implemented scope:

RQ1: Is it feasible to develop a no-code desktop software that enables direct loading of PyTorch models, execution of classification and segmentation tasks and simplifies the application and adoption of pretrained deep learning models in histopathology image analysis without programming requirement?

RQ2: Is it possible to present image classification and segmentation results, including associated confidence information and pixel-wise predictions, in a manner that is both effective and intuitive via a graphical user interface (GUI) to allow for rapid interpretation by users who are not professionally trained?

RQ3: Can the unified integration of inference history management, record retrieval (e.g., search and filtering), and one-click re-execution of prediction for records improve the usability, workflow effectiveness, and scientific reproducibility of the AI-enabled histopathology analysis system?

These questions guide the requirements and evaluation. In CP2, “direct loading” means loading supported, trusted model representations under a validated contract; it does not mean arbitrary compatibility with every file carrying a .pt or .pth extension. “Reproducibility” means restoring recorded workflow parameters under stable dependencies and files, rather than guaranteeing immutable data or identical outcomes after external files have changed.

1.2.1 Problem Statement

Three connected problems define the project. First, deployment decisions are difficult to inspect when model compatibility, preprocessing and task selection are spread across scripts or tools. ZY-Path needs to expose a simple workflow while rejecting unsupported combinations, retaining the actual model/task information and making consequential parameters visible.

Second, interpretation requires spatial and statistical clarity. A source image, a class-colour map, an overlay and a probability chart answer different questions. A user should be able to compare corresponding coordinates, see the class legend, inspect tile confidence and distinguish estimated tile coverage from segmentation pixel counts. Large images and narrower windows must remain usable without losing essential controls.

Third, completed analyses need continuity. Users should be able to retrieve a record, inspect its saved presentation, distinguish a stored result from a new run, export a report and rerun the recorded model/mode/parameters. Current preferences must not silently rewrite a historical analysis. These are specific workflow requirements; they are not assertions that competing systems lack all history or provenance capabilities.

1.3 Hypothesis

The project adopts three evaluation propositions corresponding to the original research questions. H1 is that a constrained, model-aware desktop workflow can execute classification and segmentation without users writing inference code. H2 is that explicit visual and statistical distinctions between single-image classification, tile estimates and segmentation will support usable result inspection. H3 is that restoring recorded inputs and parameters will support continuity and reduce the need to reconstruct a previous analysis manually.

H1 can be examined through executable requirements, compatible/incompatible model tests and end-to-end workflow checks. H2 requires more than correct rendering: it also requires task-based user evaluation of interpretation and perceived usability. H3 has a technical component—correct restoration and isolation from current defaults—and a user component concerning successful retrieval and rerun. This report distinguishes the available technical evidence from these untested user outcomes.

For the planned pilot, SUS is a descriptive usability measure, and a score near the commonly cited average of 68 is a contextual benchmark, not a diagnostic threshold or a guaranteed project result (Brooke, 1996; Bangor et al., 2009). No hypothesis is declared statistically accepted from development tests alone. A single-condition pilot also cannot establish superiority over another platform without an appropriate comparative design.

1.4 Project Objectives

The seven CP1 objectives are retained in substance, with the supported-model and evidence limits made explicit:

1. O1: Develop a local, no-code desktop workflow for histopathology image analysis without a running application server.
2. O2: Allow trusted, compatible pretrained PyTorch models to be selected through a graphical file browser and validated before inference.
3. O3: Support classification and segmentation with model-appropriate preprocessing, output parsing and explicit spatial sampling where applicable.
4. O4: Present class labels, probabilities, overlays, independent class legends and source/result comparisons in a readable interface.
5. O5: Export analysis presentations and available metadata as PNG images and structured PDF reports.
6. O6: Persist completed analysis records and associated artifacts locally with their actual task, model and analysis parameters.
7. O7: Support history search, date filtering, record restoration and rerun using the recorded settings.

Tiled classification, cancellation, task-specific model defaults and responsive comparison views refine these objectives rather than introduce a new training or diagnostic objective. O1's local-runtime implementation is distinguished from a distributable Windows executable: packaging remains an intended deployment step until a clean-machine build has been verified.

1.5 Scope of Work

The implemented scope comprises standard raster-image loading, local CPU inference, model validation, two explicit classification modes, compatible segmentation, result inspection, local history, cancellation and export. PNG, JPEG, TIFF and BMP are offered through the image-selection workflow. The pipeline opens the first image frame and converts it to RGB; it does not traverse a multi-resolution slide pyramid.

Single image classifies the complete selected image regardless of its dimensions. Tiled analysis uses the chosen source tile width and height even when the image fits in one tile. Consequently, a small image is not silently assigned a different inference pathway merely because a global large-image switch is enabled. Both modes use the recorded classifier contract.

The built-in classification integration expects nine ordered classes: ADI, BACK, DEB, LYM, MUC, MUS, NORM, STR and TUM. The built-in segmentation integration has

a different nine-label vocabulary. Their colour maps and statistics remain separate. Tile width and height begin at an editable 512 × 512 source pixels; their physical scale, magnification and consistency with the training crop scale remain unconfirmed.

The following CP1 exclusions remain applicable:

AI model training and optimization

ZY-Path does not provide any training loops, loss functions, optimizers, or hyperparameter tuning; users must supply fully trained models.

Dataset creation and annotation

No tools are included for labelling images, creating ground-truth masks, or organizing training datasets; the platform is inference-only.

Cloud deployment and web-based services

The system is a native desktop application, it does not offer a web interface, REST API, or any cloud-hosted functionality.

Multi-user collaboration and annotation

Designed for single-user use on a local machine, it does not support shared workspaces, simultaneous editing, or team-based annotation workflows.

Clinical diagnosis and medical certification

The software is intended solely for educational and research purposes; it must not be used for making clinical decisions or issuing medical certifications.

Two exclusions require clarification. Large raster images can now be divided into tiles and reconstructed as a class map; therefore, tile extraction and reconstruction are no longer excluded. Native gigapixel whole-slide reading, pyramid navigation, scanner-specific metadata handling and slide-level diagnostic validation remain outside scope. No Grad-CAM or other explanation method is implemented; a class overlay visualises model output and is not a causal explanation of the model.

The local-runtime scope also has prerequisites. Model files must exist, and the UPerNet path requires a locally cached verified configuration. A complete offline deployment must include these dependencies before disconnecting. The present

cloud checkout lacks the real weight files, so it cannot establish real-weight end-to-end accuracy or deployment completeness.

1.6 Research Significance

This work advances the democratization of AI for digital pathology by addressing the challenges and technical complexities related to model deployment and image analysis.

From an educational point of view, this project offers anyone with a keen interest in histopathology a chance to experiment with AI-powered histopathology analysis and to play with its functions without any programming skills. The no-code design philosophy around AI technologies fosters learning and experimentation for a wider pool of potential AI users.

The engineering contribution is a constrained integration: explicit model contracts, distinct spatial tasks, linked inspection and a record-oriented workflow are combined in a local application. The implementation demonstrates how GUI decisions—such as preserving a record's mode or separating tile coverage from global confidence—can prevent misleading interpretations of technically valid numerical outputs.

SDG 3: Good Health and Well-Being

The project contributes to the advancement of healthcare technologies by supporting AI-assisted histopathology image analysis. While not intended for clinical diagnosis, the system demonstrates how AI can assist medical image interpretation and facilitate future innovations in healthcare.

SDG 4: Quality Education

The project promotes accessible learning opportunities by enabling students and researchers to utilize AI technologies without extensive programming expertise. The no-code approach supports digital literacy and encourages broader participation in AI-assisted medical image analysis.

These links to SDG 3 and SDG 4 are motivations and potential contributions. They are not measurements of health impact, educational improvement or clinical safety. Such outcomes would require studies beyond the current software verification and planned pilot UAT.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the domain concepts, existing platforms and implementation trade-offs that inform ZY-Path. The CP1 themes—digital histopathology, pretrained models, local persistence and human-centred interaction—are retained. The CP2 review adds spatial sampling, preprocessing provenance and the distinction between software verification and model validation because these became consequential during implementation.

The review is organised thematically rather than as a claim of an exhaustive systematic review. Foundational papers, platform publications and primary software documentation are used to explain design decisions. No database search protocol, screening count or meta-analysis was performed, and none is claimed. Comparisons describe documented purposes and capabilities; commercial licensing and deployment options may change and are not treated as permanently established limitations.

2.2 Thematic Analysis of Related Research

2.2.1 Digital histopathology and the unit of analysis

Histopathology, which involves the examination of tissue sections stained with dyes like hematoxylin and eosin under a light microscope, continues to be the gold standard for diagnosing a broad range of diseases including malignancy. Pathologists make diagnoses by visual assessment of cellular morphology, cellular architecture, and the spatial organization of cells. Though, this process itself is subjective, laborious, and with potential intra-observer and inter-observer variation, particularly when dealing with borderline lesions or unusual tissue types.

Digital pathology permits image sharing and computational analysis, but a tissue patch, a larger raster region and a whole-slide pyramid are different data objects. A crop's tissue scale depends on how the image was acquired and sampled, not only its pixel dimensions. The NCT-CRC-HE-100K collection provides 100,000 nine-class colorectal tissue patches, and CRC-VAL-HE-7K provides a related external validation collection (Kather et al., 2019). These published datasets motivate the

class vocabulary, but their published results cannot be assigned to a different model without evaluation.

Published information about dataset patch size does not prove the acquisition scale of an arbitrary user image or the exact training pipeline behind a particular weight file. ZY-Path therefore exposes source tile dimensions while retaining an explicit unconfirmed-scale status. This is an implementation consequence of the literature's patch-level setting, rather than evidence that every 512-pixel crop is equivalent to a training example.

2.2.2 Classification architectures and pretrained deployment

The two main duties involved are comparable to ZY-Path. Image classification applies a categorical label, e.g. benign vs. malignant, to an input image patch. Image segmentation generates a pixel-wise mask to separate structures of interest (tumor regions, glands, cell nuclei, etc.) from background tissue. Both tasks indeed have very standard neural net architectures. For the classification, ResNet (He et al., 2016), VGG, DenseNet (Huang et al., 2017) and EfficientNet (Tan & Le, 2019) families are frequently adopted.

Dense connectivity concatenates earlier feature maps into later layers, encouraging feature reuse (Huang et al., 2017). The architecture family must be distinguished from a particular configuration. The supplied CRC training script constructs a custom DenseNet with four blocks of two dense layers; it is not a standard DenseNet-121 checkpoint and not the ResNet-50 anticipated in CP1. Architectural literature explains the design vocabulary but does not demonstrate the accuracy, calibration or CPU speed of this implementation.

Residual networks and EfficientNet remain useful comparative foundations: residual connections address optimisation in deep networks, while compound scaling studies model depth, width and input resolution (He et al., 2016; Tan & Le, 2019). ZY-Path does not implement a new architecture or run a controlled architecture benchmark. Its loader reconstructs supported architectures from metadata and checks their executable output contract.

Transfer learning can reduce the burden of obtaining task-specific models, but deploying an already trained model is not itself evidence that transfer learning was

used. In particular, the supplied training script directly instantiates DenseNet and does not load ImageNet weights. The thesis therefore describes it as an externally trained classifier, without claiming verified ImageNet fine-tuning. Model training and optimisation remain outside the GUI's scope.

2.2.3 Semantic segmentation and multiscale context

U-Net established an influential encoder–decoder approach to biomedical segmentation with skip connections that preserve spatial information (Ronneberger et al., 2015). UPerNet instead combines multiscale feature processing for scene parsing; the current built-in integration reconstructs a UPerNet model with a Swin Base backbone through Transformers (Xiao et al., 2018; Liu et al., 2021). U-Net is relevant background, not the architecture of the built-in segmentation checkpoint.

Segmentation assigns labels to a spatial output grid. When a network's prediction grid differs from the source image, a restoration rule is required. Averaging class probabilities before choosing a label and interpolating an already discrete label map are different operations. The implemented UPerNet path blends overlapping probabilities on a reduced-resolution lattice, obtains its label map and restores labels with nearest-neighbour interpolation. Its pixel coverage therefore describes the restored predicted mask; it does not establish subpixel accuracy or expert agreement at tissue boundaries.

2.2.4 Tiled classification, coverage and confidence

Spatial class maps can be constructed by assigning each crop's predicted class to its source region. This is useful for inspection but is coarser than segmentation. A class covering a large tile may occupy only part of that tile in the actual tissue. The resulting coverage must consequently be described as a tile-class estimate. Edge crops also matter: a smaller final crop must contribute fewer pixels to area coverage than a complete crop.

The model's softmax output is a normalised score over its classes. It is not automatically calibrated as the probability that an interpretation is correct (Guo et al., 2017). A high-confidence majority class across many tiles can arise from genuine image content, sampling scale, preprocessing mismatch or model/domain limitations.

ZY-Path exposes per-tile probabilities and coordinates so these possibilities can be investigated rather than using a coloured map as proof of accuracy.

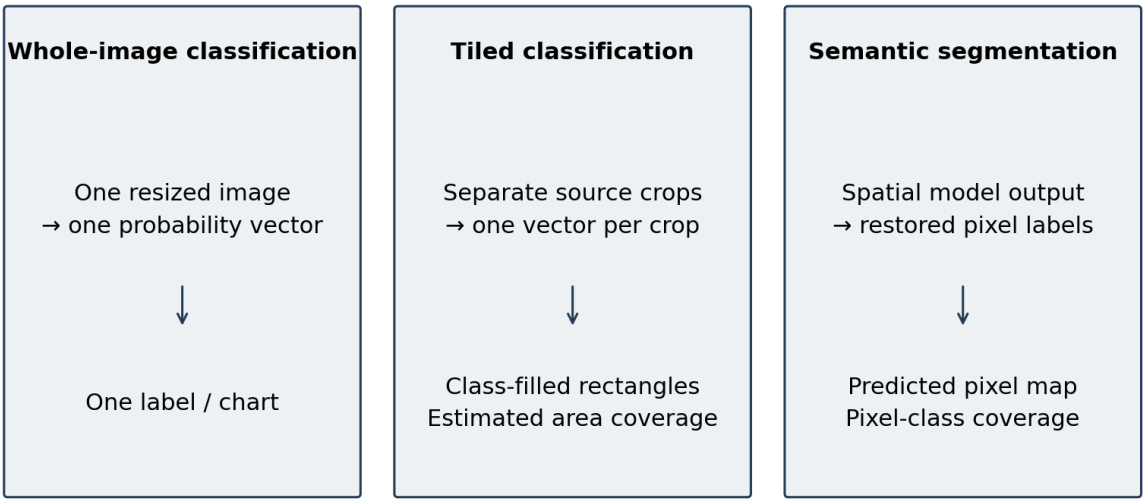


Figure 2.1. Three output meanings: whole-image classification, source-tile estimates and pixel segmentation. Author-created conceptual diagram; it contains no model accuracy results.

2.2.5 No-code interaction and transparent constraints

No-code deployment removes the need to write inference commands, but it does not remove the need for valid inputs. A .pth suffix cannot identify the network architecture, class order, normalisation or trustworthiness of a file. The interface must expose consequential choices and fail clearly when compatibility is not established. WSInfer's reusable-model approach provides an important precedent for model-aware graphical integration (Kaczmarzyk et al., 2024).

This leads to a distinction between simplicity and concealment. Whole-image and tiled analysis are explicit modes; source tile dimensions are stated in pixels; a resize backend is named precisely; and historical records retain the mode actually used. These choices make the system more inspectable while keeping network construction and tensor handling outside the user's normal workflow.

2.2.6 Persistence and reproducible workflows

SQLite provides a self-contained relational database suitable for local structured storage (SQLite, n.d.). The CP1 choice of a serverless database remains appropriate

for a single-user desktop tool. A history record can connect an image path, model path, task, parameters and saved result without requiring a separate database service.

Replaying a record nevertheless has limits. A path identifies a location, not immutable file content. A model or image may be replaced at the same path, dependencies may change and external artifacts may be removed. The present application retains useful provenance but does not persist a cryptographic identity for every input/model or a complete environment lock with each run. Accordingly, this report uses “recorded rerun” and “workflow reproducibility” rather than claiming fully immutable or universally deterministic reproduction.

2.2.7 Human-centred design and evaluation

Usability inspection and empirical user evaluation answer complementary questions. Inspection can expose inconsistent feedback or inaccessible controls, whereas task observation and questionnaires show how intended users actually experience the system (Yen & Bakken, 2012). Nielsen's heuristics support visible status, user control, consistency, error prevention and recognition rather than recall (Nielsen, 1994).

The SUS is one of the most used tools for evaluating usability in software development and in healthcare related applications. It includes ten Likert-scale questions and delivers a quantitative evaluation of the perceived usability with a very short measurement time. SUS has been validated in various application areas and allows for comparison to existing studies.

SUS provides a standard ten-item instrument, but its 0–100 score is not a percentage of tasks completed or a measure of diagnostic accuracy (Brooke, 1996). Interpretation should include sample composition, missing data and the tasks users attempted. Feature-specific ratings and qualitative feedback can explain a SUS result, but should be analysed separately rather than added to the standard SUS calculation.

2.3 Review of Existing Solutions and Platforms

2.3.1 QuPath

QuPath is an open-source digital-pathology application with image visualisation, annotation, quantitative analysis and project-oriented workflows (Bankhead et al., 2017). It already provides a graphical interface and extensibility through scripts and extensions. Scripting is relevant to advanced customisation but is not a prerequisite for every interaction. Its whole-slide and annotation capabilities exceed the current scope of ZY-Path.

The relevant difference is workflow focus. ZY-Path concentrates on selecting supported local PyTorch models, inspecting three explicitly distinguished output types and recalling individual inference records. QuPath should be viewed as a stronger general image-analysis environment, not as a GUI-deficient baseline. No timed comparison or user study against QuPath has been performed.

2.3.2 Slideflow and Slideflow Studio

Slideflow supports deep-learning development for digital histopathology, with slide processing, model training and real-time whole-slide visualisation (Dolezal et al., 2024). Its official repository explicitly describes Slideflow Studio, model-development functionality, stain-processing tools and visualisation. These documented capabilities correct the proposal's implication that Slideflow is only a command-line framework.

Compared with this broader research environment, ZY-Path intentionally excludes model training, multiple-instance learning and native whole-slide data management. Its smaller scope may simplify a specific local inference task, but that possibility remains a usability hypothesis until evaluated. A feature-rich platform is not inherently less usable; its suitability depends on the task and user population.

2.3.3 WSInfer and QuPath integration

WSInfer makes pretrained deep-learning models reusable in a pathology workflow and integrates inference with QuPath (Kaczmarzyk et al., 2024). Its emphasis on model sharing and a standardised interface is directly relevant to the compatibility and provenance requirements of ZY-Path. It also demonstrates that no-code model use is not a unique invention of this capstone.

ZY-Path differs in its standalone, record-centred presentation, explicit single/tiled classifier modes and combined local history/export controls. These are implementation choices rather than proof of a superior inference engine. WSInfer's whole-slide context and model distribution ecosystem address requirements that ZY-Path does not attempt to cover.

2.3.4 Cytomine

Cytomine supports collaborative analysis of very large images, image annotation and integration of analysis workflows (Marée et al., 2016). A collaborative web architecture is appropriate where shared images and multi-user annotation are central. It introduces infrastructure and administration considerations that differ from a single-user local desktop application.

ZY-Path trades these collaborative and large-image-management capabilities for a local application with no running application server. That trade-off is a scope choice. A local SQLite database does not replace a collaborative repository, access-control system or institutional image-management infrastructure.

2.3.5 Commercial environments

Commercial platforms such as Aiforia Create provide an alternative model-development and deployment environment (Aiforia, n.d.). Their value can include managed workflows and institutional support, while licensing, deployment and import interfaces depend on the product and agreement. This project did not obtain licensed installations or independently test their supported model formats.

Consequently, the comparison does not assert that all commercial systems prohibit custom models, require continuous internet access or lack history. The more defensible project distinction is local control over the implemented workflow and source code, within the compatibility and maintenance limits of a student-developed application.

2.4 Comparative Evaluation of Existing Systems

The comparison in Table 2.1 is qualitative and based on published purposes and primary platform descriptions. “Not evaluated” is used where a capability was not independently investigated. A feature is not marked absent simply because it was

not part of the review. No aggregate superiority score is calculated because the systems solve different problems.

Table 2.1. Scope-based comparison of pathology platforms

System	Documented emphasis	Interface/deployment	Relation to ZY-Path
QuPath	Whole-slide viewing, annotation, quantitative analysis and extensions	Graphical desktop; optional scripting	Broader image-analysis scope; advanced extension setup differs from a constrained model-file workflow
Slideflow/Studio	Model development and whole-slide visualisation	Python research environment plus graphical Studio	Includes training and WSI capabilities outside this project's scope
WSInfer + QuPath	Reusable pretrained pathology inference	Inference integrated with a desktop pathology viewer	Closest precedent for graphical model use; model-distribution and WSI focus differs
Cytomine	Collaborative large-image annotation and analysis	Web/server-oriented collaboration	Supports multi-user work excluded from ZY-Path
Aiforia Create	Commercial AI model development/deployment	Vendor-managed product options	Import, licensing and history details not independently evaluated
ZY-Path	Supported local inference, spatial inspection, record restoration and export	PyQt5 desktop with local CPU and SQLite	Narrower raster-image scope; independent accuracy, UAT and packaging validation remain pending

Two considerations follow. First, classification/segmentation support alone is insufficient for comparing workflows: users also need to understand sampling scale, probability meaning and provenance. Second, a lighter local workflow can be useful without replacing an established WSI platform. ZY-Path's evaluation should therefore test its own requirements before making comparative claims.

2.5 Identified Research Gaps and Project Positioning

The project addresses three scoped integration needs. The first is model-aware accessibility: compatible models should be usable through graphical controls while unsupported inputs are rejected with actionable messages. The second is interpretive consistency: source, overlay, class map, per-tile detail and aggregate

statistics should use the same spatial coordinates and explicit task semantics. The third is record continuity: history restoration, rerun and export should retain the analysis configuration rather than current defaults.

These needs refine the CP1 gap analysis. They do not establish the absence of similar functionality in every academic or commercial system. The project's contribution is an implemented combination tailored to individual educational/research use. It introduces no novel neural network, no clinical decision rule and no claim that classification tiles replace pixel segmentation.

The positioning also identifies an evaluation gap within this project itself. Automated tests can verify tile coordinates and metadata restoration, but they cannot establish that users understand the difference between coverage and confidence. The planned UAT therefore includes explicit interpretation tasks in addition to ordinary button completion.

2.6 Technological Infrastructure and Implementation Considerations

Python remains the implementation language because it connects the desktop GUI, tensor inference and scientific imaging libraries. The CP1 three-layer concept remains useful: presentation is implemented in ui/, inference in core/, and persistence/report generation in data/. MainWindow also coordinates saving and display, so the design is a layered desktop application rather than a perfectly isolated MVC implementation.

SQLite is a serverless, embedded, relational database engine, where the complete database (definitions, tables, indices, and the data itself) is stored in a single cross-platform file on disk. Its zero-configuration design enables the storing of structured data without having to use an external database server. This is what makes SQLite perfectly suited for local desktop applications such as ZY-Path, where inference records (image paths, model paths, prediction outputs, timestamps) must be persisted reliably in a DBA-free environment.

PyQt5 provides native windows, widgets, layouts, signals and worker threads (Riverbank Computing, n.d.; The Qt Company, n.d.). The current spatial viewer uses QGraphicsView rather than only a scaled QLabel. This allows source-coordinate transformations, linked navigation and vector grid painting after the image has been

scaled. PyTorch supplies model evaluation and inference_mode (Paszke et al., 2019); NumPy and Pillow support image/tensor conversion (NumPy Developers, n.d.; Pillow Contributors, n.d.). Matplotlib is used for the single-image classification probability chart (Hunter, 2007), while spatial overlays are generated from image arrays. fpdf2, rather than an unresolved ReportLab/fpdf2 alternative, is the implemented PDF library.

The dependency declarations specify PyQt5 5.15.x, PyTorch 2.1.x, torchvision 0.16.x, Transformers 4.38.2, Pillow 10.x, NumPy 1.26.x, Matplotlib 3.8.x, OpenCV-headless 4.11.0.86 and fpdf2 2.7.x. These are declared ranges or pins, not a complete environment lock. OpenCV is used only for the explicitly selectable classifier resize backend; Pillow bilinear remains the default. No REST API, web frontend or remote inference service is required by the implemented application.

The architecture does not make every installation automatically offline-ready. UPerNet reconstruction requests a configuration from the local Hugging Face cache with local_files_only=True. A deployment package must therefore include the verified configuration and real weights in advance. PyInstaller remains a packaging recommendation (PyInstaller Development Team, n.d.); the reviewed checkout contains no verified executable/specification or clean-Windows-machine test result.

2.7 Synthesis of Findings and Project Implications

The literature supports pretrained-model reuse, explicit task semantics and complementary technical/user evaluation. It also demonstrates that graphical pathology systems and sophisticated visualisation already exist. ZY-Path should consequently be judged on correct integration and workflow clarity rather than unsupported novelty or superiority claims.

Three implications guide implementation. Model metadata must govern preprocessing and labels. Spatial presentation must preserve coordinate meaning across Fit, zoom, comparison and restoration. Persistent records must keep the actual analysis configuration, while communicating their dependence on files and environment stability. These implications are translated into the requirements and traceability matrix in Chapter 3.

2.8 Summary

The reviewed work establishes the conceptual basis for a local inference application and clarifies its boundaries. The final implementation retains the CP1 emphasis on a no-code interface, local persistence and result export, while adding explicit tiled classification and stronger presentation/provenance controls. The next chapter explains how these design choices are implemented and how their available evidence is evaluated.

Chapter 3: Methodology

3.1 Overview of System Methodology

The project uses iterative implementation and regression-driven refinement of the CP1 design. Each iteration starts from a workflow requirement or observed defect, inspects the relevant code and data contract, implements the smallest coherent correction, and verifies both the changed behaviour and adjacent functions. Windows user reports informed several interface corrections; automated Qt tests supplied repeatable regression evidence. This is an engineering development method, not a clinical trial or a completed experimental comparison of models.

The implemented system has three logical layers. The presentation layer contains the main window, inference/history/action panels, settings and result viewers. The core layer loads models, prepares inputs, executes inference and parses outputs. The data layer stores records and creates export files. MainWindow coordinates transitions between these layers, including saving result artifacts before inserting a completed history record.

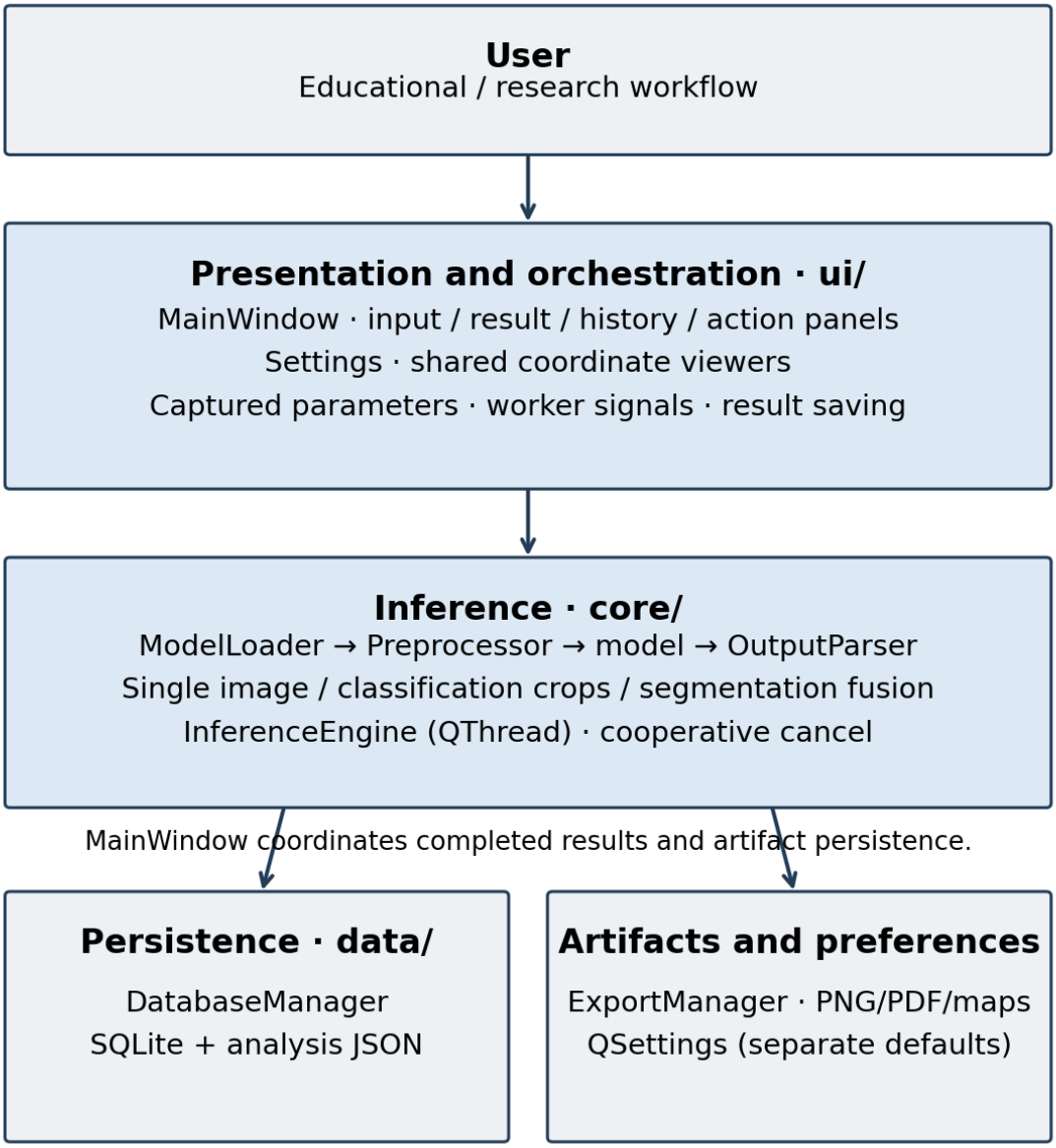


Figure 3.1. Implemented logical architecture and artifact flow. Source: author, based on the reviewed Python modules.

The software baseline for the results in this report is branch codex-ready-2026-10-15 at commit 1f3e01e7866d1b1f78103d817214f7dbb1cc5a56. The reviewed training attachment is train_crc_dn21.py, not a missing similarly named file assumed to exist in the repository. Runtime claims are based on application source and recorded tests; training design claims are based on the attachment. Neither source substitutes for real weight files or independent evaluation data.

3.2 Functional and Non-Functional Requirements

3.2.1 Functional Requirements

The requirements preserve the original model–image–inference–history workflow and add explicit spatial analysis and state-management requirements revealed during development. A requirement describes intended behaviour; its implementation and evidence status are recorded separately in Section 3.6.6.

Table 3.1. Functional requirements and acceptance conditions

ID	Requirement	Observable acceptance condition
F01	Load supported raster images	Selected input is identified and rendered; invalid files produce a clear error
F02	Load and validate trusted compatible models	Supported representations load; finite output/task/class checks reject unsupported inputs
F03	Run single-image classification	The complete selected image is processed once and yields ordered class probabilities
F04	Run explicit tiled classification	Selected width/height generate non-overlapping real crops, including partial edges
F05	Show classification statistics	Nine-class legend, per-tile label/confidence and actual-area estimates agree with stored tiles
F06	Run compatible segmentation	The segmentation model produces a restored label map, overlay and its own legend/counts
F07	Inspect spatial results	Overlay/map/original, opacity, comparison, linked navigation and appropriate grid remain aligned
F08	Save completed history	A record connects actual model/task/mode/parameters to saved artifacts
F09	Search and filter history	Keyword and inclusive local-date queries return the expected records
F10	Restore a stored result	Restoration displays the recorded result without initiating a new forward pass
F11	Rerun a record	The recorded model, image, task and parameters take precedence over current defaults
F12	Export results	PNG/PDF contain the selected presentation and available recorded metadata
F13	Cancel inference	Cooperative cancellation leaves no new completed record and preserves the previous result

ID	Requirement	Observable acceptance condition
F14	Configure task-specific model defaults	A validated local default can be saved/restored independently for each task
F15	Configure analysis and interface preferences	Mode/default selection, source tile dimensions and resize method persist with clear scope
F16	Manage records safely	Confirmed deletion removes selected records/app-owned artifacts without deleting source inputs

3.2.2 Non-Functional Requirements

Table 3.2. Non-functional requirements and verification boundaries

ID	Quality requirement	Verification approach	Present boundary
N01	Responsive interaction	Worker-thread signals; cancellation and close-lifecycle tests	A model operation may delay cancellation acknowledgement
N02	Traceable analysis	Recorded parameters, restored metadata and rerun isolation tests	File hashes and complete environment snapshots are not persisted per run
N03	Local data handling	Code-path inspection; local SQLite/artifact storage	Local files are not automatically encrypted or backed up
N04	Readable/resizable interface	Small logical screens, font/DPI simulation, theme and focus tests	Windows-native behaviour still requires final device verification
N05	Measurable runtime/resource use	Planned fixed-workload benchmark with CPU/RAM/image/tile details	No independent performance benchmark is available
N06	Predictable error handling	Missing-file, invalid-model, export/database and cancellation tests	Structural model validation is not a security or accuracy certificate
N07	Deployable offline runtime	Local configuration/weights and clean-machine packaging checks	Real weights/cache/package completeness are not verified in this checkout
N08	Maintainability and regression control	Layered modules, named tests, source revision and preserved failure reporting	No current coverage percentage is claimed

The CP1 coverage target of at least 80% on core/data modules remains an evaluation target, not an observed result. The recorded pytest runs did not produce a coverage report. Likewise, a large test count is not evidence that every hardware, accessibility or packaging requirement has been met.

3.2.3 Ethical Considerations

The application is an educational and exploratory research tool. It does not issue medical diagnoses, recommend treatment or replace professional review. Labels such as TUM and Cancerous epithelium are model categories, not verified findings about an individual patient. Softmax confidence is displayed as model output and must not be presented as a clinically calibrated risk estimate.

Only authorised, de-identified images should be used in demonstrations and UAT. Source filenames, directory names and report metadata can contain personal information even when the visible image is anonymised. Consent and institutional requirements should be checked before collecting participant data or distributing images. No ethics-approval number or participant consent is claimed without supporting records.

Local execution avoids uploading images to an inference service, but it does not automatically provide encryption, secure deletion, access control or regulatory compliance. History files, exported reports and backups remain subject to the host computer's security arrangements. Reports intended for sharing should be inspected for identifying paths and metadata.

Model trust is a further ethical and technical concern. The loader first attempts supported loading paths, including a weights-only torch.load attempt, but retains a fallback for serialized modules. Such files can execute code during deserialisation. The intended workflow therefore accepts trusted models from known sources; a file-extension filter and dummy forward pass must not be described as a sandbox or malware detector (PyTorch Contributors, n.d.).

3.2.4 Accessibility Compliance

Accessibility is addressed as a set of design and verification activities. Controls use text labels and tooltips; class legends include names and numerical values as well

as colour; the interface supports keyboard focus and Space toggling for result checkboxes; scroll areas and draggable splitters expose content in smaller windows. High-DPI sizing uses Qt logical coordinates rather than multiplying screen geometry by the device pixel ratio.

WCAG 2.2 informs the review of contrast, keyboard operation and non-colour cues, while recognising that this is a desktop application rather than a web page (W3C, 2023). No formal WCAG conformance, assistive-technology certification or complete screen-reader audit has been performed. A Windows-native keyboard/scale check, contrast audit of all text states, accessible-name review and screen-reader walkthrough remain necessary before claiming compliance.

The nine-class palette is a display aid, not a claim that all colour pairs are equally distinguishable for every user. Text legends, tile details and statistics provide redundant information. Checked, hovered and keyboard-focused result controls use stable geometry, and mouse focus no longer adds a thick frame that could obscure their usable area. These measures support accessibility without implying a completed accessibility study.

3.3 Methodologies, Tools, and Techniques

3.3.1 Model representations and executable validation

ModelLoader accepts .pt/.pth files subject to existence, non-empty-file and maximum-size checks. The maximum is 500 × 1024^2 bytes, or 500 MiB. It attempts TorchScript, serialized nn.Module objects and supported checkpoint dictionaries. Named torchvision reconstruction is limited to the architectures explicitly registered in the loader; a general state dictionary with no recognised architecture contract is not arbitrarily guessed.

The supervisor-style CRC checkpoint supplies DenseNet configuration, class labels, raw255 preprocessing and a 224 × 224 input size. The dedicated UPerNet state-dictionary path uses a verified Swin Base configuration already available in the local Hugging Face cache and a strict weight load. Missing configuration or mismatched state shapes produces an actionable validation error rather than a random replacement network.

Validation executes a float32 RGB probe of shape (1, 3, 224, 224), extracts a tensor or an object's logits field, rejects non-finite values and identifies classification output (1, C) or segmentation output (1, C, H, W). It checks executable structure, not training quality, authenticity, calibration or diagnostic fitness. Models with special input requirements, unsupported dictionaries or missing custom Python classes may still be incompatible even when their file extension is allowed.

3.3.2 Classification model and training-reference audit

The supplied train_crc_dn21.py constructs torchvision.models.DenseNet using growth_rate=32, block_config=(2, 2, 2, 2), num_init_features=64, bn_size=4, drop_rate=0 and num_classes=9. The four two-layer blocks make the precise configuration important: describing the built-in model as ResNet-50 or standard DenseNet-121 would be inaccurate.

The dataset loader reads images with OpenCV and converts BGR to RGB. The validation transform is Albumentations Resize(224, 224) followed by ToTensorV2; the dataset then converts the result to float. No divide-by-255 or ImageNet normalisation is present in that path. The training transform additionally specifies vertical/horizontal flips, hue adjustment, rotation and a random-sized crop. The reference code uses CUDA, a weighted cross-entropy objective, Adam (learning rate 0.001; weight decay 10^{-6}) and ReduceLROnPlateau. These facts describe the supplied script, not a training run reproduced in this project.

The script records training/validation metrics and supports resume, but the metric files, manifests, completed-run logs and real checkpoint are absent from the evidence package. Its default epoch count is not the verified number of completed epochs. It also checks disjoint image paths, which does not itself prove patient-level separation. Accordingly, no training curve, independent accuracy or completed epoch count is invented in Chapter 4.

The separately supplied config.py describes a three-class thyroid workflow and cannot establish the colorectal class order or source tile scale. It is excluded as evidence for the nine-class CRC contract. Neither attachment establishes the physical magnification or original-pixel dimensions of every image used to train the actual classifier.

3.3.3 Classification preprocessing and resize provenance

Preprocessor converts each input to RGB and outputs a contiguous float32 NCHW tensor. For the built-in classifier, the tensor shape is (1, 3, 224, 224) and the profile is raw255: pixel values are kept on their original approximately 0–255 scale. The optional imagenet profile divides by 255 and applies mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). It is not silently substituted for the built-in raw255 profile.

The default classification resize backend remains pillow_bilinear. The optional opencv_linear backend explicitly calls cv2.resize(..., interpolation=cv2.INTER_LINEAR), retains uint8 interpolation rounding and then converts to float32. Both single-image and tiled classification use the selected backend through the same Preprocessor implementation. The backend name is stored with the analysis so rerun does not depend on a later preference change.

The training attachment invokes Albumentations Resize without an explicit interpolation argument. Inspected library defaults are consistent with OpenCV linear, but the exact training library version is unavailable. Pillow bilinear and OpenCV INTER_LINEAR can differ in coordinate sampling, antialiasing and rounding; sharing a descriptive term such as “bilinear” is not proof of tensor identity. The implementation therefore preserves the legacy default and records the alternative, rather than changing preprocessing to improve the appearance of a particular prediction.

3.3.4 Explicit classification modes and crop geometry

Single image resizes the complete selected image and obtains one class-probability vector. Tiled analysis instead partitions source coordinates into non-overlapping rectangles. For image width W, height H and selected source tile width w and height h, the number of tiles is:

$$N = \mathrm{ceil}(W / w) \times \mathrm{ceil}(H / h). \quad (3.1)$$

At source location (x, y), the crop rectangle is [x, y, min(x+w, W), min(y+h, H)]. The right and bottom coordinates are exclusive. Each rectangle is cropped from the actual source image, independently resized to network input and classified once.

Partial edge crops contain real pixels only; they are neither padded nor replaced by the complete image or a previous crop.

For W=3552, H=3536 and w=h=512, there are seven columns and seven rows, hence 49 tiles. The final column is 480 pixels wide and the final row is 464 pixels high. The bottom-right crop contains 222,720 pixels rather than 262,144 pixels. A deterministic fixture verifies this arithmetic and that all source pixels are covered once. It does not verify the predicted tissue classes of a real image.

The choice of source crop size changes the field of view presented to a fixed 224 × 224 network. A 512-pixel source crop resized to 224 is a different sampling operation from a native 224-pixel patch, unless acquisition scale and intended field of view establish equivalence. The interface therefore states that training scale is unconfirmed and that 512 × 512 is only an editable starting value.

3.3.5 Class probabilities, maps and estimated coverage

For tile i with class logits z_i, the softmax probability for class c is $p_ic = \exp(z_ic) / \Sigma_j \exp(z_{ij})$. The tile label is $k_i = \text{argmax_c } p_ic$ and its displayed confidence is p_i,k_i. These values are retained alongside the source rectangle. The ordered nine-class colour corresponding to k_i fills that crop's source coordinates in the classification map.

Let a_i denote the actual crop area. The tile count for class c is $n_c = \Sigma_i\ 1(k_i=c)$, while its estimated covered area and ratio are:

$$A_c = \Sigma_i\ a_i\ 1(k_i=c), \qquad r_c = A_c / (W \times H). \qquad (3.2)$$

The denominator includes the complete source image, including crops assigned to background or other classes. Counts and coverage answer different questions. A small edge crop contributes one tile to n_c but only its real area to A_c. The ratios sum to one for a complete partition, apart from numerical rounding in display.

The implementation also retains an aggregate class based on area-weighted mean probabilities:

$$q_c = \Sigma_i\ a_i\ p_ic / (W \times H), \qquad k^* = \text{argmax_c } q_c. \qquad (3.3)$$

This aggregate need not equal the class with the largest r_c because probability mass and hard-class coverage are different summaries. It is not a new whole-image forward pass. The history table therefore shows tile count and largest estimated

coverage, leaving whole-image confidence blank; PDF metadata distinguishes “Aggregate tile label” and “Area-weighted mean probability”.

Table 3.3. Classification class order and fixed display palette v1

Index	Label	Tissue category	RGB
0	ADI	Adipose	(230, 159, 0)
1	BACK	Background	(210, 210, 210)
2	DEB	Debris	(86, 180, 233)
3	LYM	Lymphocytes	(0, 158, 115)
4	MUC	Mucus	(240, 228, 66)
5	MUS	Smooth muscle	(0, 114, 178)
6	NORM	Normal colon mucosa	(213, 94, 0)
7	STR	Stroma	(204, 121, 167)
8	TUM	Tumour epithelium	(120, 60, 140)

The palette is application display metadata, not evidence of a training colour convention. ClassificationTileSpec validates source/input dimensions, nine unique model classes, matching class order, matching preprocessing and nine distinct RGB colours. An incompatible class contract is rejected rather than substituted with segmentation labels.

3.3.6 Segmentation pathway and label restoration

The built-in segmentation path uses independent defaults: source tile=1000, stride=512, network input=224 and batch size=1 on CPU. These must not be confused with the user-editable 512 × 512 classification source tiles. The generic compatible segmentation path processes its prepared tensor and restores an argmax label map through OutputParser; it does not use the classifier tile statistics.

For the UPerNet path, tile_positions includes the final axis position so the complete source extent is covered. Images smaller than a source tile are reflect-padded for this segmentation pathway. Patches are resized with Pillow bilinear and ImageNet-normalised. Per-patch softmax probabilities are blended on a lower-resolution canvas using a two-dimensional Hann window with a small positive floor. The

weighted result is normalised, converted to class labels, restored using nearest-neighbour interpolation and cropped to the original dimensions.

The resulting nine-label counts use the final restored label map. Pixel coverage is class pixel count divided by W × H, including Exclude and Whitespace/Empty in the denominator. It is a statistic of a predicted mask, not an editable estimate of ground truth. The output retains its actual tile/stride/input metadata and its own colour semantics.

Table 3.4. Built-in segmentation labels and display colours

Index	Model label	RGB
0	Exclude	(0, 0, 0)
1	Cancerous epithelium	(255, 0, 0)
2	Stroma	(0, 255, 0)
3	TILs	(0, 0, 255)
4	Normal epithelium	(255, 255, 0)
5	Junk/Debris	(128, 0, 128)
6	Blood	(0, 255, 255)
7	Other	(255, 165, 0)
8	Whitespace/Empty	(255, 255, 255)

Custom-model compatibility remains constrained. Executable output validation does not guarantee semantically correct labels or arbitrary colour-map support. The built-in nine-label path is the documented reference; additional custom architectures/classes require matching metadata and explicit verification rather than assuming every segmentation model is supported.

3.3.7 Background execution and cancellation

InferenceEngine subclasses QThread and emits result_ready, error_occurred, progress_changed, status_changed and cancelled signals. ModelValidationThread performs asynchronous validation. The GUI captures the intended task and parameters before starting a worker, locks conflicting controls while busy and

updates status/progress through signals rather than running the forward pass in the GUI event loop.

Cancel sets a threading.Event. The worker checks it before/after key stages and between tile operations; it cannot interrupt a running PyTorch forward operation at an arbitrary instruction. The interface therefore distinguishes the request ("Cancelling...") from acknowledgement, keeping inputs locked until the worker exits. A late result after cancellation is rejected, and no partial completed record is saved. Closing while a worker or model-validation task is active waits for cleanup rather than terminating the thread forcibly.

3.3.8 History schema, artifacts and rerun isolation

DatabaseManager uses sqlite3 connections with parameterised statements and transaction handling. The base inference_records table stores paths, task, label/confidence where applicable, result-image path and timestamp. A companion segmentation_artifacts table stores artifact paths and analysis_json; despite its legacy name, it also carries tiled-classification and single-classification parameter metadata.

Table 3.5. Implemented persistent data structures

Structure	Principal fields	Purpose
inference_records	id, image_path, model_path, task_type, result_label, confidence, result_image_path, timestamp	Searchable completed-analysis identity and base metadata
segmentation_artifacts	record_id, result_labels_path, result_color_mask_path, result_overlay_path, analysis_json	Optional spatial artifacts and recorded analysis/presentation metadata
Classification analysis JSON	mode, resize_backend, classes/colours, source/input sizes, tiles/probabilities, counts, areas, scale_status, presentation	Preserve tile semantics and restore/reuse the actual parameters
Segmentation analysis JSON	recorded task/model, labels/colours, dimensions, tile/stride/input, counts, presentation	Restore spatial results without borrowing classification semantics

Structure	Principal fields	Purpose
QSettings	theme, current/default classification settings, per-task default model paths, reminders, window geometry	User preferences, distinct from completed record parameters

UTC timestamps are stored and converted for local display/filtering. A date range is interpreted as inclusive local days and converted to UTC query boundaries. Both date selectors and popup calendars explicitly use English, while the date format remains yyyy-MM-dd. Searches cover image/model names and do not perform visual image-similarity retrieval.

Restoring a record reads its saved artifacts and metadata without rerunning the model. Rerun is a separate operation that selects exactly one recorded analysis and uses its model, source image, task, mode, tile dimensions and resize backend. Missing files produce a clear message rather than an unannounced substitution. Current model defaults and preferences do not relabel an existing record. Deletion requires confirmation and removes selected records plus recognised app-owned result artifacts, not the original user image or model.

Saving is not a distributed transaction across image files and SQLite. Artifacts are written through the application coordinator and the record is inserted when saving succeeds; failures are reported. Filesystem errors can still leave incomplete or orphaned artifacts, and backups are not automatic. This is a maintenance/reliability boundary rather than an implemented guarantee of atomic recovery across all resources.

3.3.9 Export and local model defaults

ExportManager uses Pillow/Matplotlib for PNG and fpdf2 for structured reports. Spatial exports can include analysis JSON, class/label maps and overlays as appropriate to the recorded task; a PDF includes available source/result images and metadata. The report uses the current result's recorded identity, not whichever model happens to be selected for a subsequent run. Changing overlay opacity or view mode changes presentation, not class labels, probabilities or stored counts.

The Models settings page maintains independent classification and segmentation defaults. A candidate default is validated for the intended task before it is saved, and “restore built-in” resets that task's model preference. A missing custom default is reported as unavailable rather than silently falling back. Historical rerun still uses its recorded model path. Defaults are convenience preferences, not a migration of historical analyses to new weights.

3.4 User Interface (UI) and User Experience (UX) Design

3.4.1 Principles and information hierarchy

The interface keeps the CP1 three-region structure: input controls on the left, analysis/history in the centre and export/session actions on the right. Implementation changes replace fixed panel assumptions with draggable splitters, scrollable contents and bounded window restoration. The aim is to keep essential actions reachable while prioritising the analysis image.

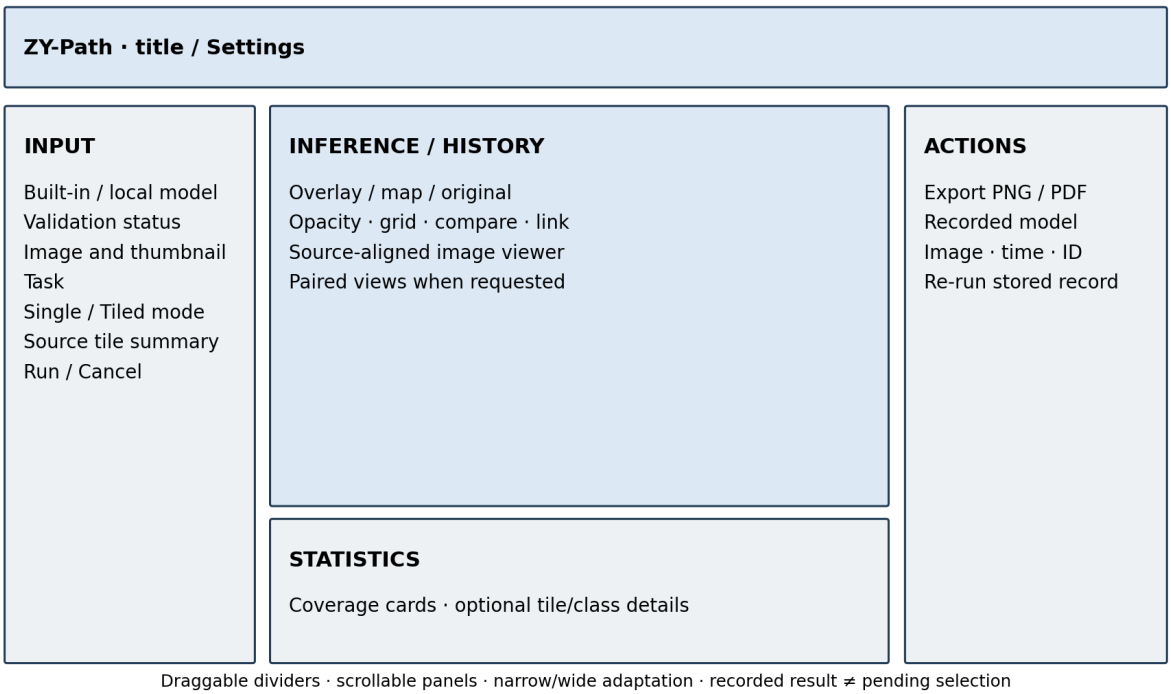


Figure 3.2. Annotated final workflow wireframe. This author-created design diagram summarises implemented regions and is not a Windows runtime screenshot.

Task and classification mode are visible before running. Status messages identify validation, inference, cancellation and completion. Completed result metadata remains distinct from a new pending selection. The design applies visibility,

consistency, recognition and user control from the usability literature, but a claim of reduced cognitive load awaits the planned user evaluation.

3.4.2 Spatial results and progressive disclosure

Tiled classification and segmentation initially hide the duplicated source preview so the main analysis view has more space. Users can select Original image or enable Compare with original. Overlay is the default spatial presentation; opacity is adjustable, and a pure class/segmentation map remains available. The classification coverage cards stay visible without requiring navigation among Image, Legend and Tiles pages; per-tile and extended class details are disclosed when needed.

In comparison mode, linked views share zoom and source-coordinate centres. Both panes use a common Fit scale rather than independently fitting dissimilar viewport widths. Narrow pages stack the pair vertically; widening restores horizontal comparison. When segmentation comparison is active, its legend stays below the images to avoid a wide-window side legend shrinking the comparison back below its horizontal-layout threshold.

The classifier grid is painted after the image with the same scene-to-viewport transform and an approximately constant logical line width. This avoids losing lines by pre-rendering a fine grid into a large image and then shrinking it. Fit, 1:1, manual zoom, resize and history restoration preserve source-coordinate alignment. The displayed scale accounts for device pixel ratio: 1:1 denotes one source pixel per physical display pixel, while window bounds are handled in Qt logical coordinates.

3.4.3 Control states, settings and help

Result toggles use a padded rounded surface. Their complete visible area—including label and padding—accepts a mouse click. Mouse focus does not leave the former thick blue frame; keyboard Tab/Backtab focus retains a thin accent border, and Space toggles the state. Constant border width prevents focus/checked states from changing the control's size.

Settings is resizable and scrollable. Preferences groups export reminders, classification mode, source tile dimensions, resize backend, separate colour references and layout reset. Selecting a mode applies to the main panel and startup default, while history restoration does not rewrite that default. Closed dropdowns

pass wheel gestures to page scrolling rather than changing the selection accidentally.

Guide & Support provides numbered sections, a fixed Contents menu, previous/next buttons and a reading-aware section indicator. System Information displays read-only runtime/library/configuration text before copying it; the copy confirmation expires after 2.5 seconds. About reports version 1.0. These features improve discoverability, but they are not a substitute for observed participant understanding.

3.4.4 Responsive windows and platform limits

Window fitting uses the current screen's availableGeometry in logical coordinates and checks restored size/position. The implementation distinguishes normal, maximised and fullscreen states and avoids continuously forcing a size during interactive resize. Scroll areas, wrapping and suitable size policies keep child widgets from imposing an oversized application minimum.

Automated checks include small logical screens, repeated narrow/wide transitions and Qt scale-factor simulation. They verify Qt-level layout and state behaviour, not Windows-native title-bar buttons, taskbar docking or multi-monitor driver behaviour. Final Windows screenshots and the manual sequence “maximise → restore → resize smaller → maximise → restore” remain part of the acceptance checklist.

3.5 Unified Modeling Language (UML) Diagrams

3.5.1 Use Case Diagram

The primary actor is an educational/research user. No administrator or clinical approval role is implemented. Model validation is included in the run workflow; tiled classification extends the classification choice when that explicit mode is selected. Restoring history and rerunning history are separate use cases because only the latter executes inference and creates a new record.

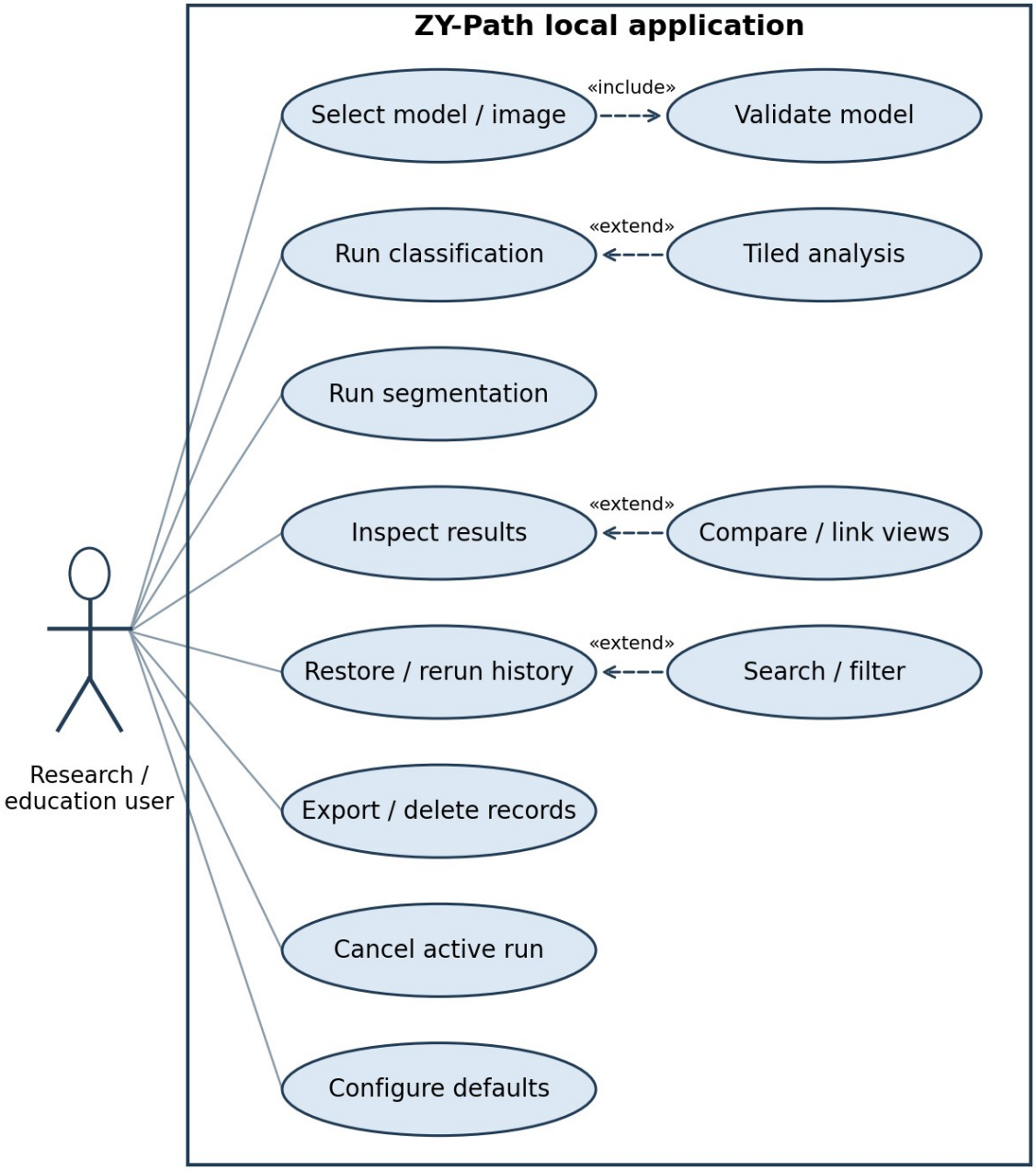


Figure 3.3. Use case diagram for the implemented local analysis workflow. File/model trust and clinical interpretation remain external responsibilities.

3.5.2 Sequence Diagram

The sequence diagram follows a successful completed run after the user selects an image, task and model. MainWindow captures parameters, starts the worker and receives progress/result signals. The worker validates and executes the pipeline; the GUI coordinator renders the result, saves artifacts and inserts a history record.

Cancellation and saving failure are alternatives, not successful completed-history paths.

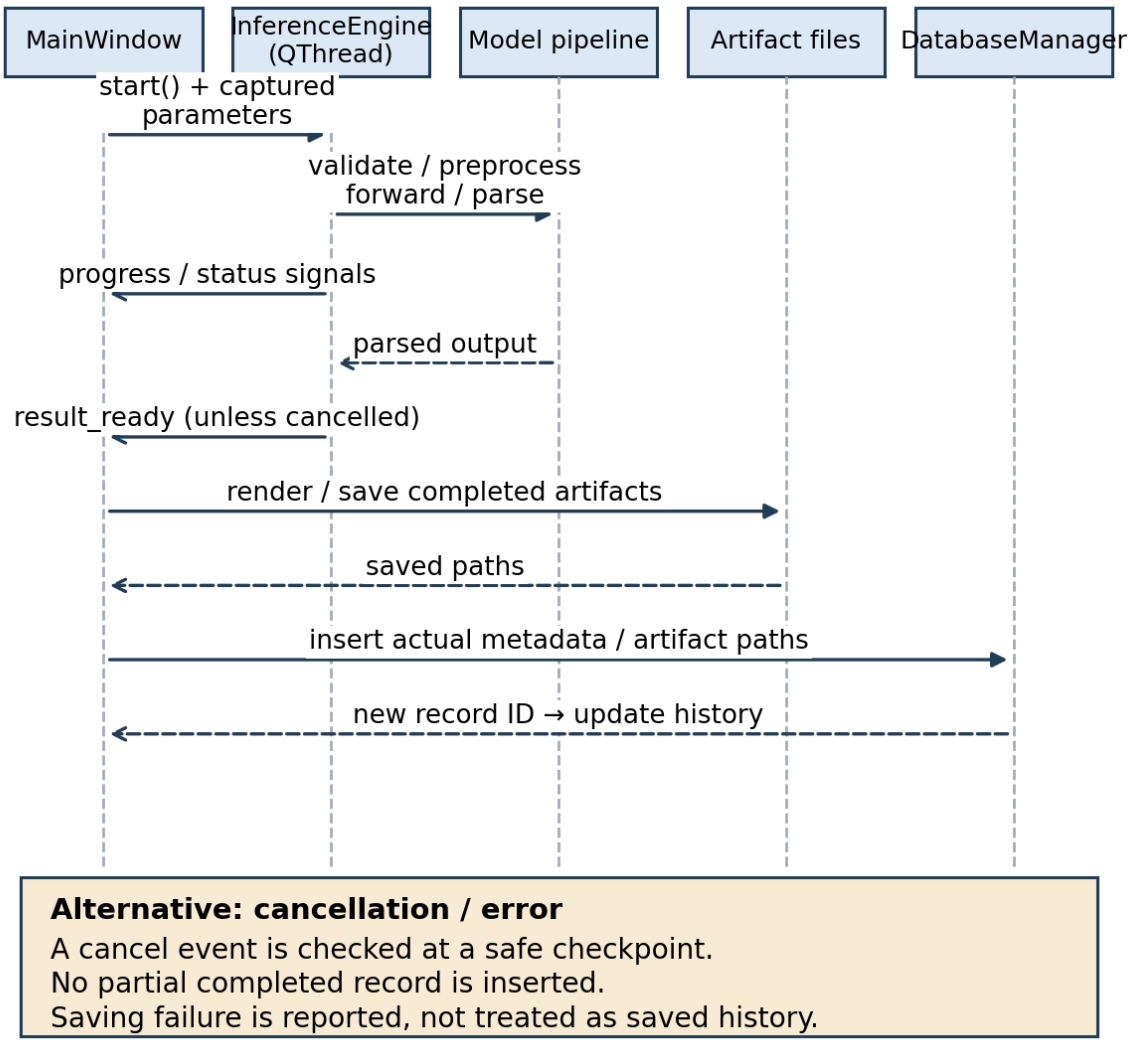


Figure 3.4. Sequence diagram for a run and its persistence boundary. Cancellation is acknowledged after a safe checkpoint; artifact-save failures must not be represented as successful saved records.

3.5.3 Class Diagram

The class diagram shows principal runtime responsibilities rather than every widget. MainWindow coordinates panels and services; InferenceEngine depends on the loader, preprocessor, parser and task-specific spatial functions. ClassificationResult and SegmentationResult share ImageComparison and ZoomableImageView. DatabaseManager and ExportManager handle persistence/export; QSettings stores preferences separately.

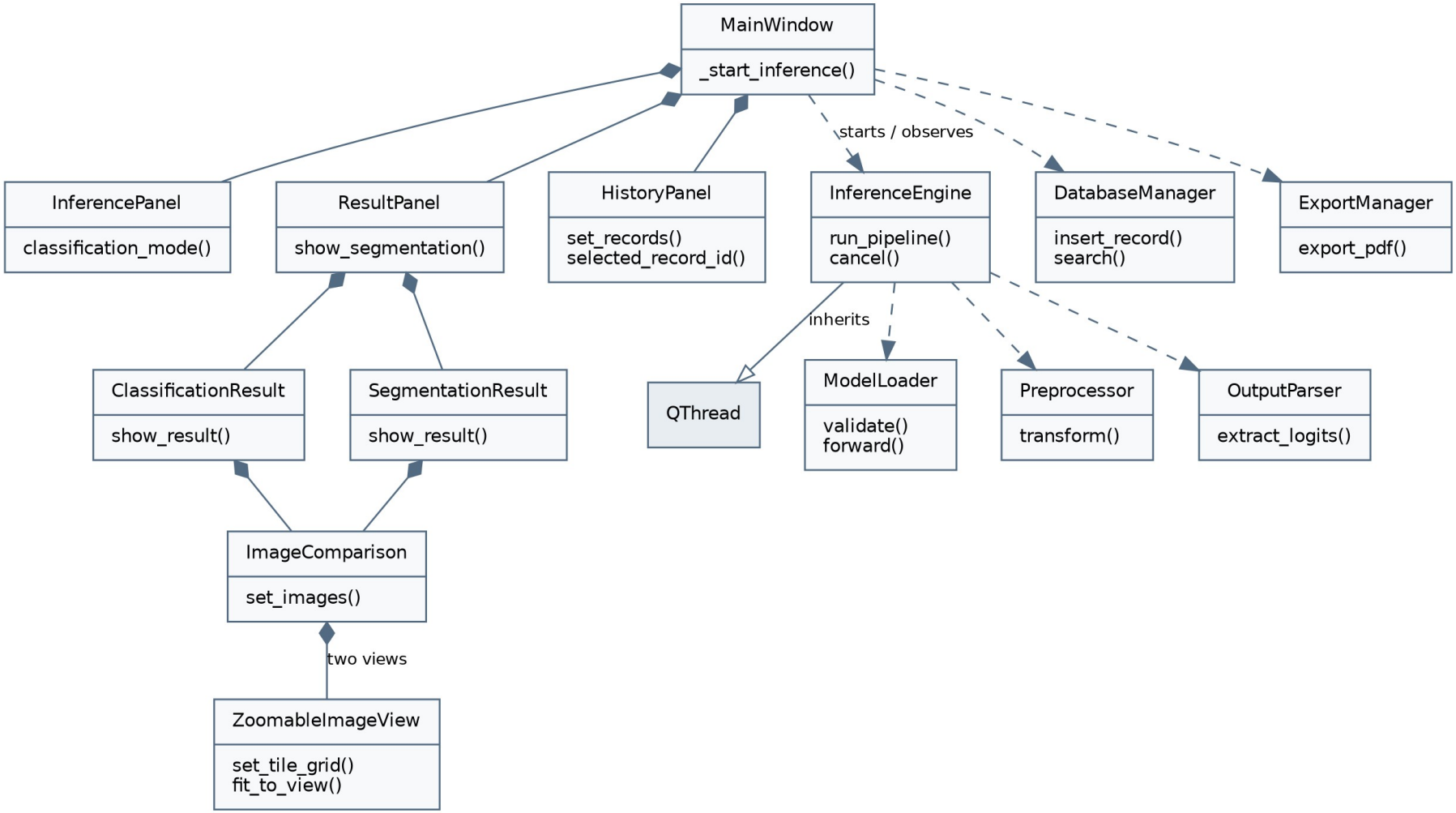


Figure 3.5. Implementation-level class diagram. Relationships represent principal ownership or dependency, not a generated inventory of every method.

3.5.4 Activity Diagram

The activity diagram highlights the explicit mode decision, distinct preprocessing/task branches and completion/cancellation outcomes. It prevents a misleading conceptual flow in which image dimensions alone choose tiled classification or a cancellation request immediately terminates a model operation.

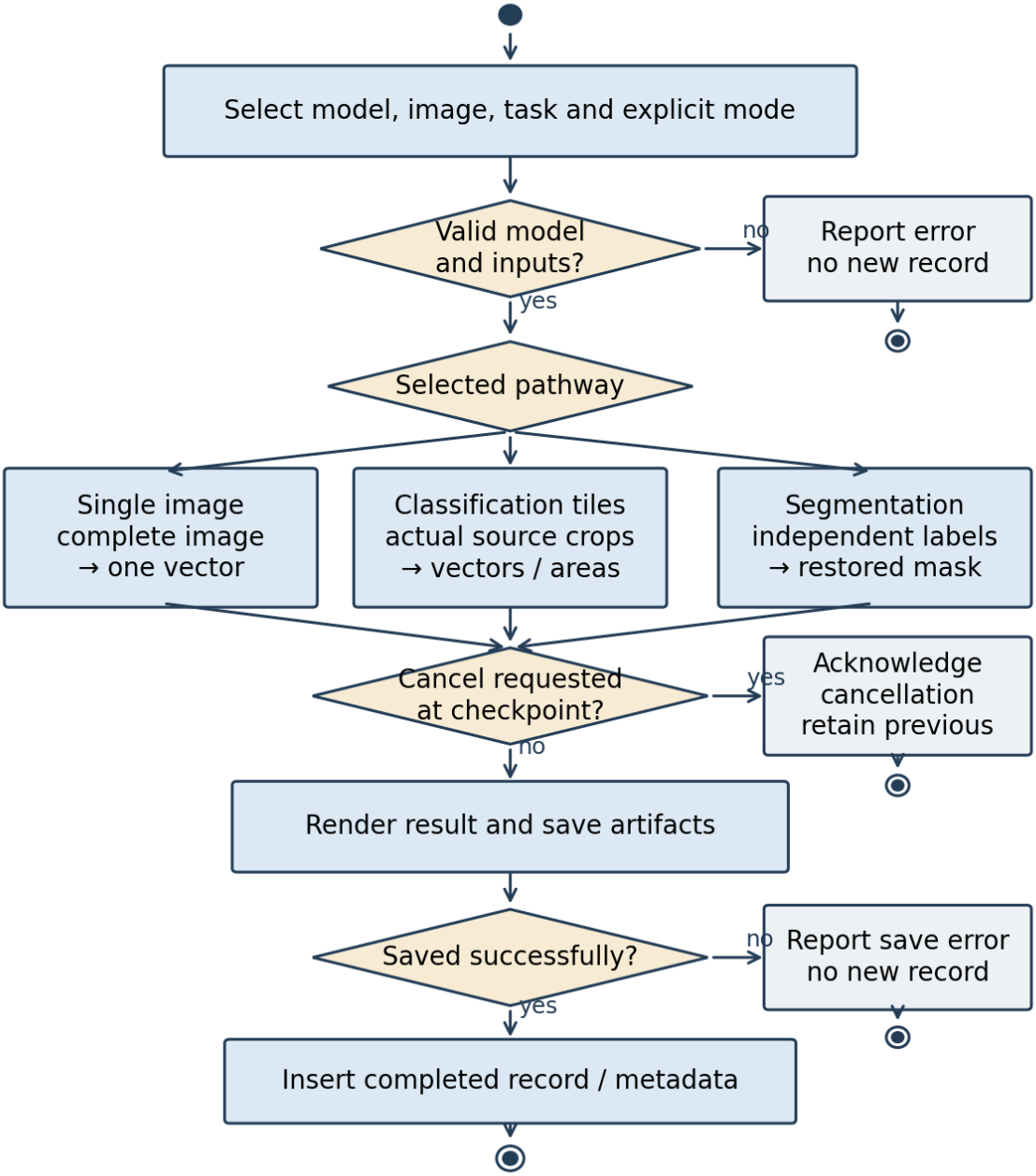


Figure 3.6. Activity diagram from parameter selection to completed persistence, with safe cancellation and error exits.

3.6 Testing Methodology

3.6.1 Development Testing

Unit testing is an independent verification of a single software component. In ZY-Path, unit testing can be applied in model loading, image preprocessing, inference execution, and SQLite operations.

Integration testing checks if there is any problem among modules after they are integrated. Since ZY-Path combines image processing, AI inference, database management, and graphical user interface (GUI) elements, integration testing verifies that data is seamlessly transferred across the entire inference pipeline.

Development tests use pytest and pytest-qt. Small in-memory networks, synthetic tensors, generated raster images, temporary SQLite databases and isolated QSettings files make failures reproducible without distributing the real large weights. These are test doubles with known behaviour, not placeholder production models. The real built-in paths are deliberately left unchanged so their missing-file failure remains visible.

Assertions check external behaviour: crop coordinates and input values, area conservation, mode routing, probability order, saved metadata, rerun isolation, cancellation outcomes and accessible controls. GUI input tests use mouse/keyboard events, not only programmatic setChecked calls. Presentation tests check that Fit, zoom and grid changes do not alter analysis data. A screenshot can support inspection but does not replace these behavioural assertions.

3.6.2 Module Testing and Fallback Handling

The loader is tested against supported representations, invalid metadata, missing/empty files, wrong shapes and non-finite logits. Preprocessor tests verify RGB conversion, profile values and resize backends. Tile tests verify unique crops, right-exclusive rectangles, partial-edge areas and agreement of a direct crop with the same preprocessing pathway. Parser tests check softmax/class mapping and mask restoration.

Database and export tests use temporary artifacts to check CRUD operations, date bounds, report metadata and error propagation. Rerun tests change current

selections/defaults to ensure they cannot overwrite recorded task or model parameters. Cancellation tests cover requests before load, at pipeline checkpoints, on the final tile and during GUI cleanup, including the rejection of a late result.

Fallback is intentionally constrained. Missing weights, missing cached UPerNet configuration and incompatible checkpoints produce explicit errors. An unsupported architecture is not replaced by a random network. A missing historical model is not replaced with the current default. The software can remain available for other operations after an error, but “graceful handling” does not mean silently obtaining a prediction from a different model.

3.6.3 Manual System Testing

Manual testing supplements automated checks on the actual Windows installation. The evidence checklist records software commit, Python/dependency versions, display scale, available screen geometry, model checksum if available, source dimensions and selected parameters. Each case records expected behaviour, actual behaviour, pass/fail, screenshot/log identifier and tester/date. Untested cases remain marked pending.

Required scenarios include startup with present/missing defaults; both classification modes; segmentation; cancel/close while busy; source/result comparison; Fit and 1:1; narrow/wide docking; maximise/restore; history retrieval and rerun after preferences change; deletion; PNG/PDF export; English calendar; and keyboard/padding clicks. The uploaded local Windows reports motivated corrections, but unstructured user observations are not recast as a completed formal acceptance protocol.

3.6.4 User Acceptance Testing (UAT)

The planned pilot retains the CP1 target of approximately 8–10 participants, subject to recruitment, consent and institutional requirements. Participants should include intended educational/research users with varied programming and pathology experience. Recruitment is purposive/convenience sampling rather than a representative clinical sample; results should be reported accordingly.

Participants receive the same orientation and authorised sample files, then attempt a fixed task sequence: load/classify an image; select tiled analysis and inspect a tile;

explain tile coverage versus confidence; compare aligned source/result regions; inspect segmentation; locate and restore history; rerun after changing current defaults; export; cancel; and configure/restore a task-specific model default. Assisted completion is recorded separately from independent completion. Cancellation is judged against the intended safe-stop outcome, not the speed of killing a thread.

After the tasks, participants complete the unmodified ten SUS items, separate feature ratings and open-ended questions. Completion time, assistance, errors and interpretation responses are recorded. There are no actual participant responses at the time of writing. Appendix B contains the consent briefing, tasks, questionnaire and recording rules; the accompanying workbook is a data-entry/scoring template, not a populated dataset.

For participant i with item responses x_ij in 1–5, the SUS score is:

$SUS_i = 2.5 \times [\Sigma(\text{odd } j)(x_ij-1) + \Sigma(\text{even } j)(5-x_ij)]$. (3.4)

A complete score requires all ten valid responses. Missing answers are not treated as zero and are not silently imputed. Report the number of valid SUS forms, mean, standard deviation and range; a median/IQR and a cautiously interpreted interval can be added for the small pilot. Feature ratings retain their own denominators and exclude “not used” responses rather than treating them as the lowest score.

3.6.5 Testing Summary

Three evidence streams are kept separate. Automated development/integration tests establish repeatable technical behaviour under controlled fixtures. Manual Windows testing establishes behaviour on the intended operating system, display configurations and real files. UAT evaluates observed task performance and user perceptions. Independent model evaluation is a fourth, distinct activity needed for predictive claims; no amount of GUI testing replaces it.

The reported test baseline is the completed recorded run for the reviewed source revision, not a new unexecuted command. The complete suite was run with --continue-on-collection-errors so available tests still executed despite one missing-script collection failure. Passing, failed and collection-error counts are reported explicitly in Chapter 4. No tests were deleted, skipped or weakened to conceal these prerequisites.

3.6.6 Requirements Traceability Matrix

The matrix connects objectives, requirements, implementation and executable evidence. “Verified with fixtures” denotes software behaviour established without the real production weights. “Pending” is used for absent model, UAT, packaging or platform evidence. A requirement can be implemented while its real-weight or user-level acceptance remains incomplete.

Table 3.6. Requirements traceability matrix

Requirement / objective	Implementation	Principal executable evidence	Status and boundary
F01 / O3	InferencePanel; image opening in InferenceEngine	test_inference_panel.py; test_inference_engine.py	Fixture-verified; final representative real files pending
F02 / O2	ModelLoader; ModelValidationThread	test_model_loader.py	Structural checks verified; real built-in file availability fails
F03 / O3	Preprocessor; OutputParser	test_preprocessor.py; test_output_parser.py; test_inference_engine.py	Fixture-verified; predictive accuracy pending
F04–F05 / O3–O4	ClassificationTileSpec; predict_classification_tiles	test_classification_tiles.py; test_classification_modes_display.py	Crop, mode and actual-area statistics fixture-verified
F06 / O3–O4	large_image_segmentation; SegmentationResult	test_large_image_segmentation.py; test_segmentation_results_layout.py	Synthetic segmentation verified; real-weight/ground-truth study pending
F07 / O4	ImageComparison; ZoomableImageView	test_linked_image_comparison.py; test_screen_fit_and_tile_grid.py	Coordinates/zoom/grid verified in Qt; Windows native confirmation pending
F08 / O6	DatabaseManager; MainWindow artifact saving	test_database_manager.py; test_main_window_inference.py	Temporary persistence verified; known real-file test fails
F09 / O7	HistoryPanel search/local-date queries	test_history_panel.py; test_result_controls_and_calendar.py	Query semantics and English calendar verified
F10–F11 / O7	Stored restoration and recorded rerun	test_main_window_rerun.py; test_classification_modes_display.py	Recorded task/model/parameters fixture-verified

Requirement / objective	Implementation	Principal executable evidence	Status and boundary
F12 / O5	ExportManager and export coordinator	test_export_manager.py; test_main_window_export.py	File/report semantics verified; final Windows report screenshots pending
F13 / O1	Cooperative cancel and close lifecycle	test_cancellation_and_model_defaults.py	Safe checkpoint and late-result handling fixture-verified
F14–F15 / O1–O3	ModelSettingsPage; SettingsDialog; QSettings	test_cancellation_and_model_defaults.py; test_preferences_interaction.py	Defaults, validation and history isolation fixture-verified
F16 / O6–O7	Confirmed record/artifact deletion	test_main_window_delete.py; test_database_manager.py	App-owned cleanup and source preservation fixture-verified
N01, N04 / O1, O4	Worker signals; logical screen bounds; scroll layouts	test_window_lifecycle_and_theme.py; test_settings_polish_and_resize.py	Qt-level evidence; native device and accessibility audit pending
N02, N06 / O2, O6–O7	Metadata validation and explicit errors	test_main_window_rerun.py; test_cancellation_and_model_defaults.py	Relevant checks pass; immutable provenance not implemented
N03 / O1	Local filesystem/SQLite; no inference service	Source inspection of core/data and utils/paths.py	Local execution established by design; security audit not performed
N05, N07–N08 / O1	Runtime/ deployment/ regression process	Recorded pytest logs; planned benchmark/package protocol	Regression evidence available; benchmark, clean package and coverage pending

This matrix is also a reporting control. A green UI test cannot change the status of missing weights, UAT or packaging to “verified”. Appendix C provides the evidence-file index, and Chapter 4 discusses the residual gaps against these requirements.

3.7 Risk Management and Mitigation Strategies

The CP1 risks concerning model compatibility, dependencies, packaging, CPU performance, recruitment and data reliability remain relevant. CP2 adds scale ambiguity and misleading spatial statistics because explicit tiled classification introduces sampling choices that a user might mistake for validated tissue

measurements. Table 3.7 distinguishes implemented controls from future verification.

Table 3.7. Risk register and residual mitigations

Risk	Consequence	Existing control	Remaining action
R1: model/preprocessing mismatch	Structurally valid but wrong input semantics	Metadata checks, explicit profiles/backends and ordered labels	Verify real checkpoint, source scale and representative tensor equivalence
R2: missing weights/cache/package dependencies	Default model cannot run offline	Clear errors; local-only UPerNet configuration request	Supply verified weights/cache and test a clean Windows package
R3: class-map/coverage misunderstanding	Tile estimates mistaken for pixel boundaries or diagnosis	Separate modes, palettes, legends and area labels	UAT interpretation task and training-scale confirmation
R4: CPU/resource limits	Long runs or unresponsive experience	Worker execution, progress, cooperative cancel	Fixed-workload time/RAM benchmark and memory-boundary study
R5: window/DPI regressions	Hidden actions, clipped text or misaligned views	Logical-coordinate bounds, scroll areas, repeated resize tests	Native Windows docking/multi-monitor verification
R6: history/artifact loss	Incomplete restore or rerun	Transactions, file checks and app-owned deletion bounds	Backup/recovery policy and stronger artifact transaction design
R7: mutable paths/dependency drift	Rerun differs from the original	Recorded task/model path/parameters and revision logs	Per-run hashes, environment manifests and migration/version policy
R8: malicious or untrusted model	Code execution during deserialisation	Trusted-source workflow; weights-only load attempt	Signed/checksummed releases and restricted supported formats
R9: limited UAT recruitment	Uncertain usability/generalisation	Prepared protocol, questionnaire and task logs	Recruit intended users and report actual sample limitations
R10: overstatement of evidence	Unsupported accuracy or clinical claims	Separate software, model, platform and user evidence	Complete independent model/UAT studies before stronger conclusions

The register does not imply that every listed mitigation is implemented. In particular, backups, signed models, immutable provenance and clean-machine packaging are future actions. Risk review should be repeated when a model, dependency, display configuration or artifact schema changes.

3.8 Summary

The methodology turns the CP1 workflow into an explicit, model-aware desktop implementation. Classification and segmentation use independent contracts; tiled classification preserves actual source areas; history restores recorded analysis settings; and the GUI supports controlled cancellation and spatial inspection. The evaluation plan separates fixture-based engineering evidence from the real-weight, Windows-native and user studies that remain necessary. Chapter 4 reports only the results supported by the available evidence.

Chapter 4: Results and Discussion

4.1 Introduction

This chapter reports the implemented software and its available verification results. The evidence baseline is source revision 1f3e01e on codex-ready-2026-10-15 and the corresponding recorded pytest logs dated 17 October 2026. Screenshots captured in the cloud are Linux/Qt development views. Where generated images or known synthetic outputs are used, they demonstrate interface behaviour only and are labelled accordingly.

The evidence does not include real model weight files in this checkout, independent classification/segmentation evaluation, completed UAT responses or a verified clean-machine Windows executable. These are material limits. The chapter therefore distinguishes implemented functionality, controlled verification, reported Windows observations and pending acceptance evidence instead of presenting all requirements as unconditionally passed.

4.2 System Implementation Results

4.2.1 Implemented workflow and scope

ZY-Path presents a local three-region interface with input/model controls, central inference/history views and export/session actions. The implemented features correspond to the functional requirements in Chapter 3: supported image/model selection, model validation, explicit classification modes, segmentation, spatial inspection, saved records, retrieval/rerun, cancellation, defaults, deletion and export.

The application remains usable through source execution on Windows, as reported by the project user. That observation is distinct from a packaged executable test. The current repository has no verified distributable build, and the real built-in files are absent from the cloud checkout. The implementation result is therefore a source-based application with substantial automated evidence and remaining deployment prerequisites.

EVIDENCE TO INSERT — Final Windows application overview

Capture the current committed application in a normal window, with readable input, result and session/export panels. Show commit/version evidence separately; redact personal paths. Use a real authorised input and identify the selected task/mode.

Figure 4.1. Final Windows application overview (evidence pending; replace this reserved area with the actual authorised capture).

4.2.2 Classification results and spatial inspection

Single-image classification retains the original whole-image label, confidence and probability-bar chart. Tiled analysis produces a source-sized class-colour map, default overlay, nine-class coverage cards and detailed per-tile labels/confidences. The source tile width and height are editable, and the main panel explicitly shows Single image or Tiled analysis before running. Class order and palette remain independent of segmentation.

Figure 4.2 is a current Qt result-view capture using a deterministic synthetic fixture. It demonstrates the grid, paired views and statistics placement. The fixture deliberately contains 49 tiles with 47 STR, one NORM and one MUC assignments. These assignments are prescribed test data, not model predictions on the coloured source pattern and not evidence of classifier accuracy.

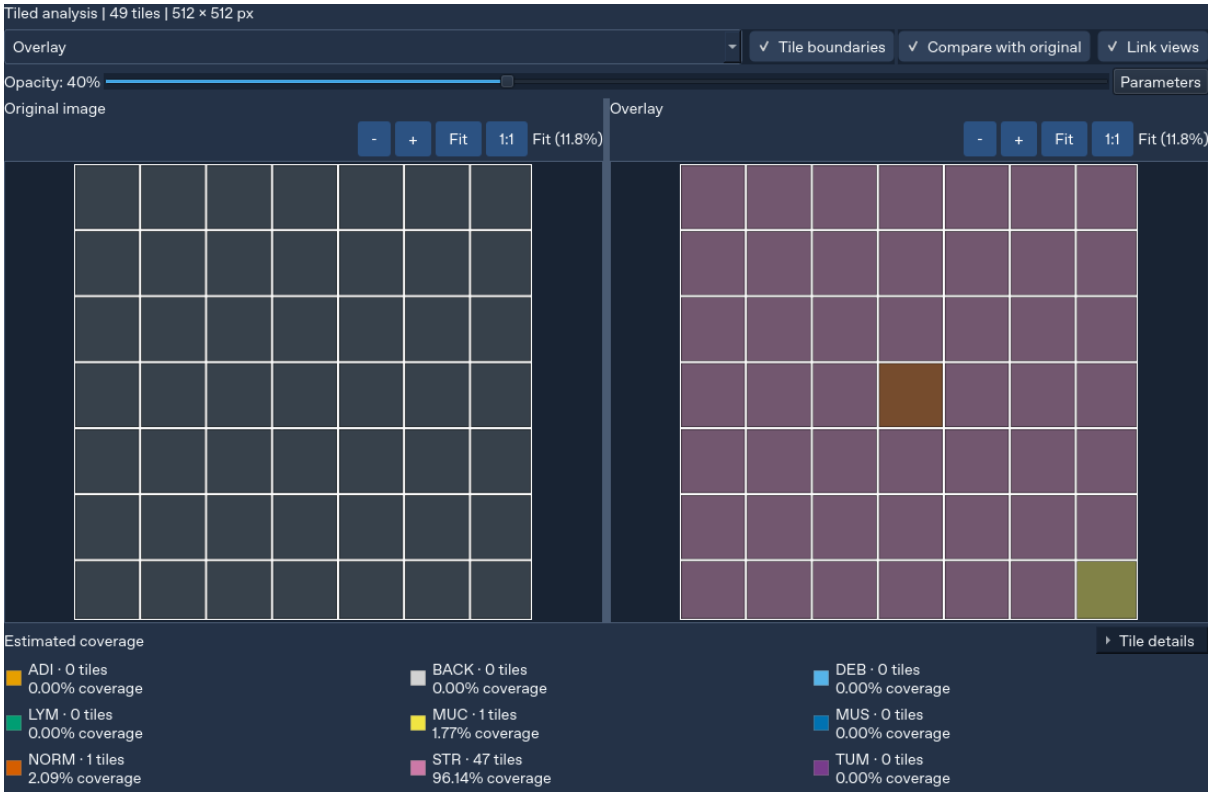


Figure 4.2. Tiled classification interface with source/result comparison and the revised result toggles. Linux/Qt capture using synthetic source/output data; not a real-weight or diagnostic result.

EVIDENCE TO INSERT — Real-weight single-image and tiled classification evidence

Provide two labelled panels from the same authorised raster image: Single image with probability chart, and Tiled analysis with mode summary, tile dimensions, nine-class legend and per-tile detail. Include image size, resize backend and model identity. Do not claim identical predictions between whole-image and crop-based modes.

Figure 4.3. Real-weight single-image and tiled classification evidence (evidence pending; replace this reserved area with the actual authorised capture).

A previously reported Windows run used a 3552 × 3536 source with 512 × 512 crops and yielded 49 tiles, predominantly STR. The count is consistent with the geometry, but the class distribution alone cannot validate the prediction. The code/test audit verifies actual source cropping, RGB/raw255 preparation, ordered softmax probabilities and area accounting under controlled fixtures. Without the real weights, source image and independent tissue annotations, it cannot decide whether that reported distribution reflects tissue content, scale/domain effects or predictive error.

4.2.3 Segmentation results

The segmentation result viewer now follows the same inspection pattern: an initial overlay without duplicate source preview, a pure segmentation map option, opacity control, optional original comparison and linked navigation. Its legend and restored-mask pixel counts use segmentation labels. Classification preferences do not alter its source tile/stride/input defaults.

The UI and large-image segmentation algorithm have controlled regression coverage, including output dimensions, class counts and blending behaviour. The supplied config.py is not evidence for this segmentation model, and the project does not infer segmentation accuracy from a matching classification region. A real-weight screenshot and a labelled evaluation set are required for stronger conclusions.

EVIDENCE TO INSERT — Windows segmentation result and aligned comparison

Capture Overlay and Compare with original using the actual segmentation model. Include the full legend, pixel-coverage label, model/session identity and matched zoom percentages. Record source dimensions and segmentation tile/stride/input metadata.

Figure 4.4. Windows segmentation result and aligned comparison (evidence pending; replace this reserved area with the actual authorised capture).

4.2.4 History, rerun and export

History stores completed records and supports keyword/date filtering, restoration and a separate rerun action. A tiled-classification row reports tile count and the largest estimated class coverage; its Confidence cell remains “-” rather than implying a whole-image confidence. Detailed per-tile confidence is available when the result is restored. The internal aggregate remains available with its explicitly different PDF terminology.

Rerun tests demonstrate that the recorded model/task/mode and analysis settings take precedence over current preferences. Restoring a record does not itself create a new prediction. The export workflow uses the displayed completed record's metadata and can retain task-specific maps/JSON together with the presentation. This prevents a newly selected model from being incorrectly attached to an older visible result.

EVIDENCE TO INSERT — History and recorded-parameter rerun

Show a real Single image row and Tiled analysis row, then restore the tiled record after changing current preferences. Capture its recorded model, mode, tile dimensions and backend. Show a separate new record after an intentional rerun.

Figure 4.5. History and recorded-parameter rerun (evidence pending; replace this reserved area with the actual authorised capture).

EVIDENCE TO INSERT — Exported report and companion artifacts

Capture an opened PDF report plus the exported files in a folder. Ensure the report identifies the actual task/model/mode; show classification tile geometry/estimated coverage or segmentation parameters as appropriate. Remove identifying source paths before submission.

Figure 4.6. Exported report and companion artifacts (evidence pending; replace this reserved area with the actual authorised capture).

4.2.5 Settings, cancellation and interface polish

Settings includes Appearance, Preferences, Models, Guide & Support, System Information and About. Model defaults are independent per task and require validation. Preferences applies the classification mode immediately while retaining a separate startup default; closed dropdowns scroll the page instead of changing selections accidentally. Guide navigation remains visible and follows the current reading section, while the system-information copy confirmation expires rather than becoming permanent content.



Figure 4.7. Implemented Guide & Support page with fixed Contents navigation and numbered guidance. Linux/Qt development capture; contact/guide content is interface evidence rather than UAT feedback.

The English calendar in Figure 4.8 was captured after setting the default locale to Chinese in a controlled run. The corresponding test opens each popup, checks English month/day labels, selects a date and verifies the unchanged date value and query signal. This is stronger evidence than merely setting a locale property without opening the widget.



Figure 4.8. English history-calendar popup under a simulated Chinese default locale. Linux/Qt capture; the selected date is test input.

Cancel inference is available during work. Tests cover cancellation at multiple checkpoints and preservation of the previous completed result. The interface waits for the current operation and worker cleanup; it does not advertise instantaneous termination. A final real-model cancellation recording remains useful because the duration of an individual model operation depends on the actual weights, workload and hardware.

EVIDENCE TO INSERT — Cancellation and per-task model defaults on Windows

Capture a running tile operation, the Cancelling state and the unchanged previous completed record after acknowledgement. Separately show a validated local default and restoring that task's built-in default. This is a multi-panel evidence figure, not a claim of instant interruption.

Figure 4.9. Cancellation and per-task model defaults on Windows (evidence pending; replace this reserved area with the actual authorised capture).

4.3 Functional Testing Results

4.3.1 Recorded automated results

The complete recorded command used pytest -q --continue-on-collection-errors --tb=short with QT_QPA_PLATFORM=offscreen and isolated writable application/configuration/cache directories. The command completed with a non-zero exit status because two known prerequisites remained missing. The final summary was “1 failed, 355 passed, 14 warnings, 1 error in 110.70s”. The duration is test-suite execution time in the cloud, not a per-image inference benchmark.

Table 4.1. Recorded automated verification outcomes at revision 1f3e01e

Run	Passed	Failed	Collection errors	Interpretation
Full pytest, continuing after collection errors	355	1	1	Full suite is not green; both missing-file issues remain visible
Result-control/history/ comparison/grid/layout regressions	83	0	0	Related controlled regressions pass
New input/locale tests at Qt scale factor 1.5	7	0	0	Simulated 150% scaling; not Windows-native proof
New input/locale tests at Qt scale factor 2	7	0	0	Simulated 200% scaling; not Windows-native proof

These runs overlap; their counts must not be added as if they represented independent test cases. A collection error can prevent a module's tests from being discovered, so a pass percentage based simply on 355/(355+1+1) would be misleading. The outcome supports the available executable cases while preserving the incomplete collection and missing production artifacts.

```
SOURCE: saved full pytest log · revision 1f3e01e
pytest -q --continue-on-collection-errors --tb=short

355 PASSED     1 FAILED     1 COLLECTION ERROR

Failure: real classification weight file is absent.
Collection error: expected CRC resume training script is absent.

Related regressions: 83 passed.
Qt scale 1.5 / 2.0: 7 passed per run (overlapping subsets).
Software verification ≠ model accuracy ≠ Windows-native acceptance.
```

Figure 4.10. Recorded pytest command/outcome excerpt and scope notes. Reproduced from saved logs for the reviewed revision; not a model-performance chart.

The failed case is tests/test_main_window_inference.py::test_default_builtin_paths_keep_real_classification_and_segmentation_models. Its assertion fails because models/crc_dn21_seed42_best.pth is absent. The segmentation real-weight file is also absent from the checkout, but this failing assertion first reports the classification path. No real model was manufactured to make this availability test pass.

The collection error occurs in tests/test_train_crc_resume.py when it attempts to read models/dn21_folder_classification/train_crc_dn21 (1).py. The separately uploaded train_crc_dn21.py is useful reference material, but it was not copied into this expected path to disguise the repository problem. No tests were deleted, skipped or reported as passing through import bypasses.

4.3.2 Challenges, resolutions and evidence

Table 4.2. Implementation challenges and verified resolutions

Challenge	Actual cause or risk	Resolution	Evidence and limit
Missing/incorrect classifier assumptions	CP1 ResNet/ImageNet description disagreed with implemented DenseNet/raw255 contract	Inspect metadata and training attachment; retain actual class order	Loader/preprocessor/tile fixtures; real predictive metrics still absent
Misleading tiled history confidence	Aggregate tile output could appear to be a whole-image label	Tile summary and estimated coverage; separate aggregate PDF terminology	History/export semantic tests; no prediction values changed
Fit grid lines disappearing	Fine rasterised grid could vanish when the large image was reduced	Paint vector grid after the image using the same transform	Source-coordinate/grid tests across Fit, zoom and resize
Comparison stayed stacked after widening	A side legend could shrink the comparison below its breakpoint	Keep segmentation legend below paired images	Repeated main-window narrow/wide regression for both tasks
Oversized/clipped windows and settings	Child minimum sizes and wrapped-text sizing constrained the layout	Logical screen bounds, scroll areas and height-for-width handling	Small-screen/font tests; native Windows verification remains pending
Tight focus box and unreliable label/padding clicks	Global focus border and native checkbox hit region differed	Stable rounded surface; full-rectangle hit area; keyboard-only focus cue	Mouse edge/text/padding, disabled and Tab/Space tests in both themes
Accidental preference changes by wheel	Closed combo boxes consumed scroll as selection input	Explicit selection; wheel forwarded to page scrolling	Popup/keyboard/wheel integration tests
Unclear cancellation/default ownership	Late worker results or current defaults could affect a record	Cooperative stop/late-result rejection; per-task defaults and recorded rerun	Cancellation/default/rerun tests with temporary artifacts
Calendar followed local language	Date popup inherited locale-dependent text	Explicit English locale for editors and calendars	Actual popup/day selection under Chinese default locale

The resolutions address reproducible implementation defects or ambiguity. They do not use segmentation as classification ground truth, change a model to force expected tissue proportions or hide errors by reducing the test scope. The traceability matrix reflects these limits: F04–F05 and F07 have strong controlled

evidence, while model accuracy, clean packaging and empirical usability remain separate open items.

4.3.3 Manual and predictive validation status

Windows user observations were valuable because offscreen Qt tests had not exposed every native layout problem. Their role was to trigger diagnosis and regression cases, not to establish a completed clinical or usability study. The current evidence package still requires a documented final native run at the reviewed revision, with normal/maximised windows, narrow/wide docking, representative display scales and real model/input artifacts.

There is no verified independent accuracy, macro-F1, confusion matrix, calibration metric, segmentation Dice or IoU for the integrated real models. The proposal's statement of accuracy above 94% cannot be retained as a project result. Published NCT results describe the published experiments; the training script's metric calculations describe intended measurement; neither supplies this project's missing evaluation outputs.

The supplementary evaluation checklist specifies model checksum, source dataset/manifests, patient/slide-level separation, preprocessing/backend, source scale, per-class support and independently generated metrics. These details are prerequisites for attributing any future number to the actual deployed model.

4.4 User Acceptance Testing Results

UAT has not yet been conducted or supplied. This section is structured for the actual results and includes explicit fields rather than simulated participants, fabricated percentages or invented quotations. The instrument and scoring workbook are complete, but an empty template is not an empirical finding. Until data are collected, H2 and the user-performance aspect of H3 remain untested.

4.4.1 Participant Demographics

Table 4.3. UAT participant profile — awaiting actual responses

Characteristic	Required result	Status
Recruited/consented/completed participants	[TO COMPLETE: actual counts and exclusions]	Not collected

Characteristic	Required result	Status
Participant role	[TO COMPLETE: role counts, with actual denominator]	Not collected
Pathology experience	[TO COMPLETE: predefined experience groups]	Not collected
Programming/AI experience	[TO COMPLETE: experience groups]	Not collected
Prior use of pathology tools	[TO COMPLETE: actual distribution]	Not collected
Testing devices/display scales	[TO COMPLETE: actual configurations]	Not collected

The proposed 8–10-person target is a recruitment plan, not the sample size. Participants should be described with anonymous IDs and aggregate characteristics. Unnecessary personal data should not be collected. If the sample mainly comprises computing students rather than pathology users, that must be stated as a limitation of interpretation-focused findings.

4.4.2 Quantitative Results

Table 4.4. UAT quantitative summary — awaiting valid data

Measure	Reporting rule	Result
Valid SUS responses	All ten items in 1–5	[TO COMPLETE]
SUS mean and standard deviation	Score each participant before averaging	[TO COMPLETE]
SUS median, IQR and range	Report alongside the small-sample mean	[TO COMPLETE]
Independent task completion	Independent successes / actual attempts	[TO COMPLETE]
Assisted completion	Report separately from independent success	[TO COMPLETE]
Critical errors and assistance	Count using the predefined task rules	[TO COMPLETE]

No comparison with the SUS benchmark is possible before valid responses exist. A future confidence interval should include the number of complete forms and

assumptions, and should not be used to suggest population-level certainty from a small convenience sample. Missing-data and exclusion rules must be applied before inspecting the desired conclusion.

4.4.3 Feature Usage and Ratings

Table 4.5. Feature ratings — awaiting actual use and responses

Feature	Required usage/result evidence	Result
Single-image workflow	Actual attempts, completion and ease rating	[TO COMPLETE]
Tiled map/legend/tile details	Actual use, interpretation correctness and rating	[TO COMPLETE]
Comparison and linked navigation	Alignment task completion and rating	[TO COMPLETE]
Segmentation inspection	Use of overlay/map/legend and rating	[TO COMPLETE]
History and rerun	Recorded-mode task completion and rating	[TO COMPLETE]
Export, cancel and defaults	Completion, assistance and feature-specific rating	[TO COMPLETE]

Each feature requires its own valid-response denominator. “Not used” is a separate response and must not be converted into a dissatisfaction score. A task successfully completed with assistance should not be labelled an independent success. Interpretation answers should be compared with the prepared semantic answer key rather than agreement with an unverified tissue prediction.

4.4.4 System Performance and Acceptability

Participant-reported responsiveness should be reported separately from instrumented runtime. The questionnaire asks whether progress/cancel states were understandable and whether the application was acceptable for the stated educational/research task. It does not ask participants to certify diagnostic safety or model accuracy.

[TO COMPLETE: actual per-task timing distributions, device/CPU/RAM details, observed freezes/crashes if any, perceived responsiveness ratings and acceptance

responses. Distinguish model loading, single-image inference, tiled inference, saving and export.]

The saved 110.70-second pytest duration is not suitable for this subsection as an inference-time result. Similarly, a single reported 0.6-second Windows prediction, without a fixed protocol and hardware/context, cannot establish a general latency claim.

4.4.5 Qualitative Feedback Analysis

Open-ended responses will be coded against a documented scheme, initially covering navigation, interpretation, responsiveness, accessibility, history/export and suggested improvements. New themes may be added if the responses justify them. The analysis should retain a short audit trail from anonymous response to code to theme, include contrary cases and avoid claiming saturation from a small pilot.

[TO COMPLETE: actual theme counts, anonymised quotations with consent, supporting participant IDs and the changes accepted or deferred. No participant quotation is supplied before collection.]

The Windows feedback already used during development is reported in Section 4.3 as informal defect evidence. It is not combined with future UAT survey responses as if both came from the same sampling protocol. This separation keeps design iteration useful without overstating the formality of the current usability evidence.

4.5 Discussion

4.5.1 Achievement of Research Objectives

Table 4.6. Objective achievement and outstanding evidence

Objective	Implemented outcome	Evidence-based assessment
O1: local no-code workflow	PyQt5 application, background CPU work and cancel	Feasibility supported with controlled models; clean offline package not verified
O2: model loading/validation	Supported formats, metadata reconstruction and probe checks	Structural integration verified; real built-in availability incomplete
O3: inference tasks	Whole-image/tiled classification and segmentation	Algorithms/parameters tested; independent real-model accuracy pending

Objective	Implemented outcome	Evidence-based assessment
O4: result inspection	Charts, overlays, legends, spatial detail and linked comparison	Qt-level behaviour verified; UAT and final native checks pending
O5: export	PNG/PDF with task-specific metadata/artifacts	Temporary-artifact/report tests pass; final demonstration screenshots pending
O6: persistent records	SQLite and saved analysis/presentation artifacts	Controlled persistence verified; immutable provenance/backup not implemented
O7: retrieval/rerun	Search, local-date filter, restore and recorded rerun	Recorded-parameter semantics verified; user-efficiency advantage untested

The most defensible achievement is implementation feasibility under a constrained contract. The system now exposes choices that the proposal either assumed or left implicit, including the distinction between source tile size and network input, backend-specific resizing and tile coverage versus global confidence. These are substantive engineering outcomes even though they do not establish diagnostic performance.

H1 is supported in the controlled software setting. The correctness components of H3 are also supported by restoration and rerun tests. H2 and claims about reduced time/cognitive load require actual users, while deployment-level acceptance requires the missing artifacts and native checks. The report therefore avoids calling every objective fully validated.

4.5.2 Comparison with Existing Solutions

ZY-Path is narrower than QuPath, Slideflow and Cytomine in whole-slide, annotation, training and collaborative functionality. Its value lies in a focused local arrangement of model-aware inference, spatial presentation and persistent per-run records. WSInfer provides a clear precedent for accessible pretrained-model use; the contribution here is the particular desktop workflow and its tested state semantics, not the invention of no-code pathology inference.

No comparative trial, installation-time study or task-time benchmark against another platform was carried out. It is therefore inappropriate to claim that ZY-Path is faster, more accurate, more usable or uniquely reproducible. A future comparison should

use matched tasks and intended users, document the configuration of each tool and include the capabilities that the current project omits.

4.5.3 Limitations and Constraints

First, the system depends on pretrained PyTorch models provided by users or default model and does not include model training functionality.

Third, collaborative annotation and multi-user access are not supported because the system focuses on standalone desktop deployment.

Fourth, prediction performance is dependent on the quality, architecture, and training data of the supplied AI models.

Finally, the system is intended for educational and research purposes only and should not be considered a replacement for professional medical diagnosis.

Additional CP2 limitations are material. Real classification and segmentation weights are missing from the cloud environment, and the resume-test collection depends on an absent script. The attached training code does not identify the original source scale or supply completed training/evaluation logs. The editable classification tile size is exploratory, and anisotropic resizing of non-square edge crops can alter morphology presented to the model.

Large raster tiling does not provide native gigapixel WSI streaming. Full-size source arrays, class maps and restored masks still consume memory, and the UPerNet probability canvas has its own resource cost. GPU inference, batched classification and whole-slide physical-scale extraction are not established runtime features. CPU timings depend on the actual model and hardware; none are generalised from the test suite.

Structural validation cannot guarantee semantic class correctness, calibration or file safety. The generic custom-model workflow is limited to supported input/output conventions and implementation-tested label handling. Full pickled modules may need their defining Python classes and carry deserialisation risks. There is no universal architecture loader, clinical validation or secure sandbox.

History references mutable filesystem paths and is not a complete provenance archive. It does not guarantee that the bytes at a path are unchanged. There is also no implemented automatic backup, multi-user access control or atomic transaction

encompassing all external artifacts. A completed record improves traceability but is not a certified clinical audit trail.

Finally, offscreen Qt and DPI simulations do not prove final Windows-native docking, window-manager or assistive-technology behaviour. The latest controls/calendar changes have repeatable input tests and screenshots, but a final native acceptance pass is still required. No UAT responses, independent metrics, coverage percentage or clean-machine executable evidence are supplied.

4.5.4 Implications for Future Development

Future work should first close the evidence gaps rather than add unrelated features. Confirm model provenance and training/source scales, restore the missing test prerequisite, evaluate real models on independently separated data and collect the planned user study. These steps determine whether the implemented workflow is fit for its intended educational/research use.

Stronger provenance should then add immutable file identifiers, software/dependency manifests and versioned analysis schemas. A transactional artifact manifest and backup/recovery workflow would improve persistence robustness. Verified packaging should include model/configuration licensing, required local caches and a clean Windows test. Native accessibility/contrast checks and screen-reader feedback should inform further UI refinement.

Only after these foundations should the project consider physical-scale-aware WSI reading, memory-bounded streaming, acceleration or explainability. Each extension changes the validation burden. A classifier heatmap must remain distinguishable from segmentation, and any explanation view must remain distinguishable from measured model validity.

4.6 Summary

The implementation delivers a substantial local inference workflow and controlled evidence for its crop geometry, state handling, inspection and persistence semantics. The recorded full-suite outcome is 355 passed, one failed and one collection error; the unresolved prerequisites remain explicit. UAT, independent model accuracy, final native acceptance and clean packaging are incomplete, so the

conclusions are limited to demonstrated software behaviour and a prepared evaluation plan.

Chapter 5: Conclusion and Future Work

5.1 Conclusion

This capstone has developed ZY-Path from a CP1 proposal into a source-based desktop application for educational and exploratory histopathology image analysis. It combines compatible pretrained-model loading, CPU inference, graphical result inspection, local history and export through a PyQt5/PyTorch/SQLite implementation. The original objective of reducing the need to write inference code is addressed through a constrained graphical workflow rather than unrestricted model compatibility.

The implementation adds explicit single-image and tiled classification, separate segmentation semantics, linked source/result inspection, task-specific model defaults and cooperative cancellation. In particular, tiled analysis retains actual crop coordinates and areas, exposes its sampling settings and labels coverage as an estimate. History and rerun preserve the recorded task/model/mode instead of silently applying current preferences. These details connect interface behaviour to the meaning of the numerical result.

Controlled verification supports many of these implementation claims. At the reviewed revision, 355 tests passed, one real-file availability test failed and one missing-script collection error remained. Related controls/history/layout regressions and simulated DPI checks passed. These outcomes should not be converted into a claim of complete deployment readiness or clinical accuracy: the real model artifacts, independent model metrics, UAT responses and final Windows acceptance evidence are not included.

The research questions are consequently answered at different levels. A model-aware local GUI workflow is technically feasible and implemented. The recorded restoration/rerun logic supports workflow continuity under stable files and dependencies. Whether the interface improves users' interpretation, satisfaction or efficiency remains an empirical question for the prepared UAT. The project's conclusion is therefore a demonstrated engineering integration with clearly identified evaluation limits.

5.2 Future Work and Recommendations

The immediate priority is evidence completion. Supply the verified built-in weights and cached configuration; reconcile the absent resume-test script with the intended repository layout; capture final Windows runs at the reported commit; and perform independent model evaluation with known patient/slide separation, source scale and preprocessing. No software test should be removed to make this process appear complete.

The second priority is user evaluation. Run the planned task-based pilot with intended users, score valid SUS responses correctly, separate assisted from independent completion and analyse interpretation errors as well as preferences. Revise the interface in response to actual findings, then rerun the relevant regressions and report which changes improved observed tasks. A later controlled comparison could investigate workflow advantages against an established platform without assuming the result in advance.

The third priority is reproducibility and deployment. Add file checksums, dependency manifests and versioned metadata to saved records; define artifact transaction/recovery behaviour and backups; package the application with all permitted resources; and test it on a Windows machine without the development environment. Security and model-licensing review should accompany distribution, especially where serialized model objects remain supported.

Longer-term work may include memory-bounded WSI reading with physical-scale metadata, optional acceleration, calibration/uncertainty analysis and carefully validated explanation views. Such work would expand the project's scope and require new requirements, datasets and tests. It should not be described as an implemented outcome of this version.

5.3 Research Contributions

The first contribution is an implemented local workflow connecting compatible PyTorch inference to a graphical desktop interface and persistent per-analysis records. It demonstrates the feasibility of integrating these components without a running application server, while preserving explicit deployment prerequisites.

The second contribution is a practical separation of inference semantics in the interface and stored metadata. Whole-image classification, tile-class coverage and segmentation pixel coverage remain distinguishable. Source-crop geometry, edge areas, resize choices, class order and the unconfirmed training scale are exposed rather than hidden behind a visually plausible output.

The third contribution is a tested approach to inspection and state continuity: shared coordinate transforms, linked views, vector tile boundaries, responsive layouts, recorded rerun settings and safe cancellation preserve meaning across user actions. The regression cases document defects that purely static or offscreen screenshots had not been sufficient to assess.

The fourth contribution is an auditable evaluation package. Requirements are connected to source modules and named tests; missing prerequisites remain failures/errors; and a complete UAT instrument, scoring template and screenshot/evaluation checklist support subsequent evidence collection. These are software-engineering contributions and reusable evaluation materials, not a novel neural architecture or a clinically validated diagnostic method.

5.4 Summary

ZY-Path implements the central CP1 workflow and extends it with explicit spatial analysis and stronger state management. Its present contribution is best understood as an inspectable research/education application supported by controlled software evidence. Completing independent predictive evaluation, UAT, native acceptance and verified packaging will determine the strength of future claims about usability, generalisation and deployment readiness.

References

Aiforia (n.d.) Aiforia Create. Product information. https://www.aiforia.com/aiforia-create

Bangor, A., Kortum, P. and Miller, J. (2009) ‘Determining what individual SUS scores mean: Adding an adjective rating scale’, Journal of Usability Studies, 4(3), pp. 114–123.

Bankhead, P. et al. (2017) ‘QuPath: Open source software for digital pathology image analysis’, Scientific Reports, 7, article 16878. https://doi.org/10.1038/s41598-017-17204-5

Brooke, J. (1996) ‘SUS: A “quick and dirty” usability scale’, in Jordan, P.W., Thomas, B., Weerdmeester, B.A. and McClelland, I.L. (eds.) Usability Evaluation in Industry. London: Taylor & Francis, pp. 189–194.

Dolezal, J.M., Kochanny, S., Dyer, E., Ramesh, S., Srisuwananukorn, A., Sacco, M., Howard, F.M., Li, A., Mohan, P. and Pearson, A.T. (2024) ‘Slideflow: Deep learning for digital histopathology with real-time whole-slide visualization’, BMC Bioinformatics, 25, article 134. https://doi.org/10.1186/s12859-024-05758-x

Guo, C., Pleiss, G., Sun, Y. and Weinberger, K.Q. (2017) ‘On calibration of modern neural networks’, Proceedings of the 34th International Conference on Machine Learning, Proceedings of Machine Learning Research, 70, pp. 1321–1330. https://proceedings.mlr.press/v70/guo17a.html

He, K., Zhang, X., Ren, S. and Sun, J. (2016) ‘Deep residual learning for image recognition’, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778. https://doi.org/10.1109/CVPR.2016.90

Huang, G., Liu, Z., van der Maaten, L. and Weinberger, K.Q. (2017) ‘Densely connected convolutional networks’, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 2261–2269. https://arxiv.org/abs/1608.06993

Hunter, J.D. (2007) ‘Matplotlib: A 2D graphics environment’, Computing in Science & Engineering, 9(3), pp. 90–95. https://doi.org/10.1109/MCSE.2007.55

Kaczmarzyk, J.R., O’Callaghan, A., Inglis, F., Gat, S., Kurc, T., Gupta, R., Bremer, E., Bankhead, P. and Saltz, J.H. (2024) ‘Open and reusable deep learning for pathology with WSInfer and QuPath’, npj Precision Oncology, 8, article 9. https://doi.org/10.1038/s41698-024-00499-9

Kather, J.N. et al. (2019) ‘Predicting survival from colorectal cancer histology slides using deep learning: A retrospective multicenter study’, PLOS Medicine, 16(1), e1002730. https://doi.org/10.1371/journal.pmed.1002730

Litjens, G. et al. (2017) ‘A survey on deep learning in medical image analysis’, Medical Image Analysis, 42, pp. 60–88. https://doi.org/10.1016/j.media.2017.07.005

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S. and Guo, B. (2021) ‘Swin Transformer: Hierarchical vision transformer using shifted windows’, Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 10012–10022. https://arxiv.org/abs/2103.14030

Marée, R. et al. (2016) ‘Collaborative analysis of multi-gigapixel imaging data using Cytomine’, Bioinformatics, 32(9), pp. 1395–1401. https://doi.org/10.1093/bioinformatics/btw013

Nielsen, J. (1994) ‘Enhancing the explanatory power of usability heuristics’, Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, pp. 152–158. https://doi.org/10.1145/191666.191729

NumPy Developers (n.d.) NumPy user guide. https://numpy.org/doc/stable/user/

Paszke, A. et al. (2019) ‘PyTorch: An imperative style, high-performance deep learning library’, Advances in Neural Information Processing Systems, 32. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

Pillow Contributors (n.d.) Pillow documentation: Image processing and resampling. https://pillow.readthedocs.io/en/stable/

PyInstaller Development Team (n.d.) PyInstaller documentation. https://pyinstaller.org/en/stable/

PyTorch Contributors (n.d.) torch.load; inference_mode; serialization semantics. PyTorch documentation. https://pytorch.org/docs/stable/

Riverbank Computing (n.d.) PyQt5 reference guide. https://www.riverbankcomputing.com/static/Docs/PyQt5/

Ronneberger, O., Fischer, P. and Brox, T. (2015) ‘U-Net: Convolutional networks for biomedical image segmentation’, in Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015, pp. 234–241. https://doi.org/10.1007/978-3-319-24574-4_28

SQLite (n.d.) Zero-configuration. SQLite documentation. https://sqlite.org/zeroconf.html

Tan, M. and Le, Q.V. (2019) ‘EfficientNet: Rethinking model scaling for convolutional neural networks’, Proceedings of the 36th International Conference on Machine Learning, Proceedings of Machine Learning Research, 97, pp. 6105–6114. https://proceedings.mlr.press/v97/tan19a.html

The Qt Company (n.d.) Qt 5 documentation: QThread, QScreen, QGraphicsView and high-DPI support. https://doc.qt.io/qt-5/

W3C (2023) Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation, 5 October 2023. https://www.w3.org/TR/WCAG22/

Xiao, T., Liu, Y., Zhou, B., Jiang, Y. and Sun, J. (2018) ‘Unified perceptual parsing for scene understanding’, Proceedings of the European Conference on Computer Vision, pp. 418–434. https://arxiv.org/abs/1807.10221

Yen, P.-Y. and Bakken, S. (2012) ‘Review of health information technology usability study methodologies’, Journal of the American Medical Informatics Association, 19(3), pp. 413–422. https://doi.org/10.1136/amiajnl-2010-000020

Appendices

Appendix A: Project Timeline and Planning

A.1 Proposal baseline

The following planning tables are retained from CP1 as historical proposal baselines. Their week labels describe the plan at that stage; they are not retrospectively presented as actual implementation dates or proof that every planned evaluation was completed. In particular, the proposed accuracy study, UAT and executable packaging still require the evidence identified in this report.

Table A.1. Original CP1 planning baseline, retained from the submitted proposal

CP1 Week	Phase Theme	Key Activities	Deliverables
3	Project Initiation	Finalize project title; articulate the no-code core concept; complete and submit the Student Assessment Form (SAF) for supervisor approval.	Approved SAF with registered title
4	Problem Identification	Identify three core gaps in existing platforms; draft the initial system flowchart; analyse QuPath, Slideflow and WSInfer; tabulate the feature comparison.	Draft problem statement and initial flowchart
5	Background Research	Collect and summaries ≥10 peer-reviewed articles covering digital pathology, deep learning, no-code AI and clinical usability; draft Chapter 1.1–1.2.	Annotated bibliography (≥10 articles)
6	Objective Definition	Define three research questions and seven specific objectives; delineate in-scope and out-of-scope items; articulate significance and SDG 3/4 alignment.	Completed Chapter 1

CP1 Week	Phase Theme	Key Activities	Deliverables
7	Literature Extension	Add five further articles (total ≥15); focus on PyTorch deployment, SQLite management and HCI heuristics; refine flowcharts into Figures 3.1 and 3.2; draft the three-layer architecture.	Expanded bibliography and detailed design diagrams
8	Semester Break	Cognitive digestion of collected literature; informal reading; mental preparation for the intensive writing phase.	None
9	Chapter 2 Completion	Synthesize all reviewed literature into coherent Chapter 2 sections; finalize and justify the development toolset (Python 3.10, PyQt5, PyTorch, SQLite, Matplotlib, fpdf2 and PyInstaller).	Completed Chapter 2 and technology justification table
10	Methodology Documentation	Document Chapter 3, including architecture, class diagram, model constraints, preprocessing, database schema and use cases UC1–UC4; formalize testing and evaluation methodology; prepare the initial Gantt chart.	Completed Chapter 3 and draft CP1 work plan
11	CP2 Planning	Draft a detailed CP2 work plan defining coding sprints, testing phases, evaluation schedules and thesis milestones; draft the abstract (≤300 words).	Draft CP2 work plan and abstract draft
12	Plan Finalization	Refine the CP2 plan based on Chapter 3 complexity; add explicit testing and debugging buffers; finalize the abstract incorporating supervisor feedback.	Finalized CP2 work plan and finalized abstract

CP1 Week	Phase Theme	Key Activities	Deliverables
13	Integration	Merge CP1 and CP2 plans into a unified Gantt chart; cross-check deliverables against research questions; standardize Harvard citations; verify figure/table numbering and heading hierarchy.	Fully formatted draft document
14	CP1 Submission	Compile Chapters 1–4, abstract and all appendices into a single PDF; submit via eLearn before the deadline; corresponds to Go/No-Go Gate A.	CP1 submission (Gate A passed)

Table A.2. Original CP2 planning baseline, retained from the submitted proposal

CP2 Week	Development Layer	Key Activities	Milestone
1–2	Core Architecture	Configure the Windows 10/11 environment (Python 3.10, VS Code, Git/GitHub and virtual environment); implement the ui/, core/ and data/ package structure; develop ModelLoader with probe-tensor validation and Preprocessor with a standard torchvision pipeline (Resize 224×224, ToTensor, ImageNet normalisation and unsqueeze).	Functional project skeleton with passing unit tests

CP2 Week	Development Layer	Key Activities	Milestone
3–4	Presentation Layer	Build the PyQt5 GUI: MainWindow with QSplitter (three regions); InferencePanel (built-in toggle, QFileDialog, upload thumbnail, Classification/Segmentation radios and Run Inference); ResultPanel (classification bar chart and segmentation overlay); HistoryPanel (QTableWidget, search and date filters); wire the signal-slot mechanism.	Fully navigable GUI shell with all panels interactive
5–6	Inference Layer	Develop InferenceEngine (QThread subclass) to execute the background inference pipeline: ModelLoader.load() → Preprocessor.transform() → torch.no_grad() forward pass → OutputParser.parse_cls()/parse_seg(); integrate the built-in ResNet-50 fine-tuned on NCT-CRC-HE-100K (nine-class colorectal tissue dataset), with automatic loading on startup.	Functional end-to-end inference pipeline
6–7	Data Layer	Implement DatabaseManager (sqlite3) to create the inference_records table with complete metadata; develop parameterised LIKE search and date-range filtering with defensive try/except handling; build ExportManager with export_png() (Matplotlib, 150 DPI) and export_pdf() (fpdf2 A4 report).	Persistent history management and professional report generation

CP2 Week	Development Layer	Key Activities	Milestone
7–8	Integration & Refinement	Validate the four core use cases (UC1–UC4) end-to-end; stress-test signal-slot communication across layers; refine the UI according to Section 3.4, including the clinical color palette, WCAG 2.2 Level AA contrast, tooltips and progress indicators.	Stable, accessible application ready for formal testing (Gate B)
9–10	Testing & Validation	Develop a comprehensive pytest suite covering ModelLoader, Preprocessor, OutputParser, DatabaseManager and ExportManager, targeting ≥80% coverage; validate the built-in ResNet-50 on an NCT-CRC-HE-100K test subset (>94% accuracy); test ≥3 custom PyTorch architectures for compatibility.	Comprehensive test report with validated accuracy benchmarks
11	Usability Evaluation	Recruit 8–10 participants, including pathology students, biomedical researchers and clinical technologists; conduct heuristic evaluation based on Nielsen's ten heuristics, a SUS questionnaire (0–100 scale) and semi-structured think-aloud interviews.	Completed SUS scores and qualitative interview transcripts

CP2 Week	Development Layer	Key Activities	Milestone
12	Results & Packaging	Analyse SUS scores against the benchmark of 68; perform thematic analysis; verify UC4 reproducibility; package the application with PyInstaller (--onefile --windowed); test on clean Windows 10/11 virtual machines without Python installed.	Validated evaluation report and deployable executable (Gate C)
13	Thesis Completion	Write Chapter 4, covering implementation outcomes, SUS results, accuracy benchmarks and gap validation; write Chapter 5, covering the conclusion, answers to the three RQs, contributions, limitations and future work, including WSI support, model training, multi-user support, XAI, cross-platform deployment and a model repository.	Complete thesis draft
14	Final Submission	Proofread and format the complete thesis; compile the final PDF; prepare the executable and supplementary materials; submit all deliverables via eLearn.	Final submission concluding the project lifecycle

A.2 Implementation and outstanding milestones

Table A.3. Evidence-based milestone status

Milestone	Available evidence	Current status
Core interface/inference/persistence integration	Reviewed ui/, core/ and data/ modules with regression tests	Implemented under supported contracts
Linked original/result views	Source history including revision 206b51f and current tests	Implemented; final native capture pending

Milestone	Available evidence	Current status
Cancellation and independent model defaults	Revision 12633fb and lifecycle/default tests	Implemented with cooperative-stop limits
Settings/history semantics	Revisions 643099b and 562ce39	Implemented and regression-tested
Settings guide and responsive comparison refinement	Revision e058354	Implemented and tested with fixtures
Full-area result toggles and English calendar	Revision 1f3e01e	Current documented baseline
Real-weight evaluation and complete test prerequisites	Required artifacts absent from the cloud checkout	Pending
Formal UAT and Windows acceptance record	Prepared instruments; no completed dataset supplied	Pending
Clean-machine packaged distribution	No verified executable/specification supplied	Pending

A.3 Completion sequence

Before final submission, first confirm the cover fields and research-document requirements, then capture current Windows evidence and assemble authorised model/data artifacts. Resolve the missing repository prerequisite without bypassing its test. Next conduct independent model evaluation and the consented UAT pilot, update Chapter 4 with actual results, and revise Chapter 5 only to the extent those results justify. Finally update contents/figure/table fields, check citations and screenshots, and validate the packaged application if distribution is claimed.

Appendix B: User Acceptance Testing Details

B.1 Participant information and consent briefing

Study purpose: evaluate the usability of ZY-Path for educational/research image-analysis tasks. The study does not assess a participant's clinical competence or establish model diagnostic accuracy. Participation is voluntary. Participants may stop at any time or decline an optional question. Any recording requires separate permission; no identifiable patient image should be used.

Before starting, confirm institutional requirements and any required approval. Allocate an anonymous participant ID. Record the application revision, device/display scale and prepared sample/model identifiers. Explain that some predictions may be incorrect and that the participant should evaluate the workflow rather than infer a clinical diagnosis. Agree how anonymised responses and any permitted quotations will be stored and used.

Consent fields: participant ID [blank]; consent to participate [yes/no]; consent to anonymised quotation [yes/no]; consent to screen/audio recording [yes/no/not requested]; date [blank]; researcher initials [blank]. Do not collect a name where an anonymous ID is sufficient. A participant who declines should not be entered as a completed response.

B.2 Background questions

Record role (e.g., biomedical/pathology student, researcher, educator, other); pathology-image experience (none, occasional, regular); programming/AI experience (none, basic, intermediate/advanced); previous pathology-tool use (yes/no; optional tool name); and testing device/display scale. Age grouping is optional and should be collected only if needed for the approved study. Do not imply that all recruited users are qualified pathologists.

B.3 Standard task script and observation rules

Table B.1. UAT tasks and success criteria

Task	Participant instruction	Observer success criterion
T01	Load the supplied image and run Single image with the approved model	Correct image/model/mode and completed output identified without coding
T02	Select Tiled analysis, set the specified source width/height and run	Correct explicit mode and source dimensions are used
T03	Find a tile's class/confidence and explain coverage	Identifies actual tile detail; states coverage is a crop-class area estimate, not pixel ground truth
T04	Compare a selected region with the original and use Fit/zoom	Same source region remains aligned; participant can return to Fit
T05	Inspect the supplied segmentation result	Distinguishes segmentation labels and pixel coverage from classification tiles

Task	Participant instruction	Observer success criterion
T06	Search/date-filter history and restore the nominated record	Correct stored record shown without initiating inference
T07	Change a current preference, then rerun the stored record	Recorded model/mode/parameters used; new record distinguished from restored one
T08	Export the nominated completed result	Correct PNG/PDF and actual task/model metadata identified
T09	Start the supplied workload and cancel it	Recognises request/acknowledgement; no partial completed record saved
T10	Save a supplied validated default and restore the built-in default	Correct task's preference changes; other task/history remain unaffected

For each attempted task record start/end or elapsed seconds, outcome (independent success, assisted success, unsuccessful, not attempted), assistance count, critical error and a brief observation. Define a stop rule consistently before testing. An intended cancellation is successful when safe acknowledgement and record preservation occur, not when the model stops instantly. If a prepared file is unavailable, record an environment-blocked attempt rather than blaming the participant or silently substituting another task.

The interpretation answer key is semantic: source tiles are resized to network input; confidence is a model score; tile coverage uses actual crop areas; segmentation has independent labels; and stored rerun uses recorded settings. It must not require agreement with an unverified predicted tissue category.

B.4 System Usability Scale questionnaire

After completing the tasks, ask participants to rate the following standard SUS statements from 1 (Strongly disagree) to 5 (Strongly agree). Keep their order and wording consistent; analyse these ten items separately from the project-specific questions (Brooke, 1996).

8. I think that I would like to use this system frequently.
9. I found the system unnecessarily complex.
10. I thought the system was easy to use.
11. I think that I would need the support of a technical person to be able to use this system.
12. I found the various functions in this system were well integrated.

13. I thought there was too much inconsistency in this system.
14. I would imagine that most people would learn to use this system very quickly.
15. I found the system very cumbersome to use.
16. I felt very confident using the system.
17. I needed to learn a lot of things before I could get going with this system.

B.5 Feature-specific ratings and open questions

Rate each feature used from 1 (Very difficult/unclear) to 5 (Very easy/clear), with a separate Not used option: model/image selection; mode/tile settings; legend/statistics; tile details; original comparison/linked views; segmentation inspection; history/rerun; export; cancellation/progress; and settings/help. Report denominators for each feature. These ratings do not modify the SUS score.

Ask: Which part was easiest? Which part was confusing? Was any statistic or colour meaning unclear? Could you recover from an error or cancel safely? What would you change first? Would you use this system for the stated educational/research task (yes, maybe, no), and why? Do not ask a leading question that assumes the application is better than another system.

B.6 Scoring, missing data and reporting

The accompanying workbook contains blank participant, task, SUS and feature-rating sheets. SUS scoring requires ten numeric integers in 1–5 and applies Equation 3.4. A blank/incomplete form remains unscored. In task results, independent completion and assistance are separate outcomes; feature “Not used” is excluded from numerical ratings. Store anonymous identifiers consistently across sheets, document exclusions and retain the unmodified raw responses separately.

For the pilot, report actual N, recruitment method, device conditions, SUS distribution, completion denominators, assistance/errors and qualitative themes. If the study lacks a comparator, describe perceived usability rather than claiming causal improvements in efficiency. Add de-identified quotations only with consent and never fill empty Chapter 4 tables using generated participants.

Appendix C: Implementation and Evidence Index

C.1 Source baseline

Repository: ziaozhuanxing/ZY-Path. Branch: codex-ready-2026-10-15. Reviewed revision: 1f3e01e7866d1b1f78103d817214f7dbb1cc5a56. The thesis artifact preparation did not change application source or push a new revision. The supplied CRC training script and unrelated three-class config are external reference attachments, not newly installed production models.

Table C.1. Principal evidence sources

Evidence	Source	Supports
E01	core/model_loader.py	Supported representations, size limit, metadata/probe checks, UPerNet reconstruction
E02	core/preprocessor.py; train_crc_dn21.py attachment	RGB/raw255/input size, selectable resize, reference transforms
E03	core/classification_tiles.py	Crop geometry, edge areas, nine-class palette, counts and aggregate probabilities
E04	core/large_image_segmentation.py; core/inference_engine.py	Independent segmentation defaults, fusion/restoration and worker cancellation
E05	ui/main_window.py; data/database_manager.py	Artifact/record creation, recorded rerun, local data paths
E06	ui/classification_result.py; ui/segmentation_result.py; ui/image_comparison.py; ui/zoomable_image_view.py	Spatial views, legends, linked coordinates and vector grid
E07	ui/settings_dialog.py; ui/model_settings.py; ui/guide_view.py	Preferences, per-task defaults and reading-aware help
E08	ui/result_check_box.py; ui/theme.py; ui/history_panel.py	Full-area toggle input, focus styling and English calendar
E09	data/export_manager.py; tests/test_main_window_export.py	PNG/PDF and recorded task/model metadata
E10	Saved full/related/scale pytest logs	Reported counts, exit outcome and known missing-file failures

C.2 Evidence interpretation

The full log establishes what executed and what did not. The related/scale logs are subsets and are not additive. Generated images and known synthetic outputs establish geometry, display and state behaviour; they do not establish tissue accuracy. Previous Windows reports establish observed issues and one local launch, but do not supply the final formal acceptance log for all features. The pending-evidence checklist identifies what still needs to be collected.

Appendix D: Manual Windows Acceptance and Screenshot Checklist

D.1 Native acceptance record

For each case record actual date, tester, application commit, Python/version, Windows build, CPU/RAM, display scale, available screen geometry, sample/model identifier, expected outcome, actual outcome and evidence filename. Use Pass, Fail, Blocked or Not run. Do not mark a case Pass because a Linux offscreen equivalent passed.

Table D.1. Final Windows acceptance cases

Case	Action	Expected evidence
W01	Launch and restore previous geometry	Normal window fits current available screen and retains usable actions
W02	Maximise, restore, shrink, maximise, restore	Native states and normal geometry behave correctly without forced fullscreen
W03	Dock narrow, inspect comparison, restore wide	Vertical/horizontal adaptation works without Reset; views remain aligned
W04	Fit, 1:1, zoom and resize a 49-tile result	Complete aligned 7 × 7 grid; 480/464-pixel edge dimensions recorded
W05	Click toggle labels, padding and indicators; use Tab/Space	One transition per click; mouse/keyboard focus styles appropriate
W06	Open both history calendars	English month/weekdays; correct selected dates and filtering
W07	Restore/rerun after changing current defaults	Recorded model/mode/parameters retained; restore itself does not rerun
W08	Cancel a real workload and close while busy	Safe acknowledgement/cleanup; no partial completed record

Case	Action	Expected evidence
W09	Scroll/maximise Settings at common display scales	All content reachable, no artificial trailing height or clipped controls
W10	Export, reopen PDF/PNG and inspect companion files	Correct task/model/statistics; files readable; no unwanted identifying data
W11	Validate/save/restore defaults per task	Clear invalid/missing-file handling and task independence
W12	Move required files and attempt restoration/rerun	Actionable missing-file errors; no silent model substitution

D.2 Required figure replacements

Figures 1.1 and 4.1, 4.3–4.6 and 4.9 explicitly reserve the required evidence. The accompanying Chinese capture guide gives step-by-step framing and filenames. Figures 4.2, 4.7 and 4.8 are labelled development captures; retain that label or replace them with equivalent current Windows captures and update the caption's environment/source description. Screenshot dates, model identity and software revision must correspond to the actual captured run.

Appendix E: Independent Model Evaluation Protocol

E.1 Classification

Before evaluation, record the actual checkpoint checksum, architecture/configuration, class order, preprocessing profile, backend/library versions and input dimensions. Identify the labelled dataset version, patient/slide grouping, source image scale and any overlap with training or tuning data. File-path disjointness alone is not evidence of patient-level independence.

For single-image classification, compute a confusion matrix, per-class support, precision, recall and F1, macro-F1 and accuracy on the specified independent set. State whether images are native patches or resized larger regions. If tiled analysis is evaluated, define its ground-truth unit and sampling scale separately; an image-level class label cannot validate every tile or a pixel-coverage map. If calibration is claimed, supply an explicit calibration protocol and metric rather than interpreting softmax confidence as calibration.

E.2 Segmentation

Use authorised independently annotated masks with the same class vocabulary. Record the original spatial scale, label alignment, ignored/background policy, tile/stride/input settings and restoration method. Compute per-class IoU/Dice and aggregate values with clear denominator/exclusion rules. Classification outputs must not serve as segmentation ground truth, and a visually pleasing overlay does not substitute for expert-labelled masks.

E.3 Runtime and deployment

Benchmark a predefined set of source dimensions and tile settings on a recorded CPU/RAM/OS environment. Separate cold model loading, preprocessing/inference, artifact saving and export. Repeat runs under a fixed protocol and report appropriate distribution summaries and peak memory. The whole pytest duration must not be used as per-image inference time. A clean-machine package test should verify required weights/configuration, file permissions, offline operation and both tasks before executable deployment is described as complete.